

Bridge: High-Order Taint Vulnerabilities Detection in Linux-based IoT Firmware

Jiaqian Peng^{†‡}, Puzhuo Liu^{*§¶}, Yicheng Zeng^{†‡}, Kai Cheng^{*†}, Yongji Liu[†], Yun Yang[†], Hongsong Zhu^{†‡}

[†]*Institute of Information Engineering, Chinese Academy of Sciences*

[‡]*School of Cyber Security, University of Chinese Academy of Sciences*

[§]*Ant Group*

[¶]*Tsinghua University*

Emails: {pengjiaqian, zengyicheng, chengkai, liuyongji, yangyun, zhuhongsong}@iie.ac.cn, liupuzhuo.lpz@antgroup.com

Abstract—The rapid proliferation of IoT devices has led to a surge in security incidents stemming from vulnerabilities in firmware. IoT device security is critical, as failures can result in privacy breaches, system downtime, and life-threatening situations. While taint analysis has been a standard approach for detecting vulnerabilities, the increasing complexity of modern IoT devices has introduced high-order taint vulnerabilities that traditional methods cannot address. These vulnerabilities often require the coordination of multiple requests and components, making detection particularly challenging. This paper presents **Bridge**, a novel approach to detecting high-order taint vulnerabilities in IoT firmware. **Bridge** operates in three stages: identifying entry points (i.e., the initial handlers corresponding to different action requests) and taint source functions (i.e., functions for parsing user-controllable data), constructing binary dependency graphs for tainted data propagation, and constructing action dependency graphs to trace taint vulnerability triggering control relationships. Extensive evaluation on 44 real-world firmware samples demonstrates that **Bridge** outperforms state-of-the-art tools, detecting 1,168 true positive vulnerabilities including 566 high-order vulnerabilities. Moreover, among the results, 90 vulnerabilities (including 45 high-order vulnerabilities) have been confirmed by vendors (CVE/PSV) and pose a threat to device security, including remote code execution and denial of service.

1. Introduction

With the widespread adoption of IoT devices, security incidents stemming from vulnerabilities in firmware have become increasingly prominent. It is estimated that the global number of IoT devices will reach nearly 30 billion by 2030 [1], with applications spanning smart homes, medical devices, and industrial control systems. Consequently, the security of IoT devices has garnered significant attention. Security vulnerabilities in firmware are often the root cause of user privacy breaches [2, 3], device downtime [4], and even life-threatening incidents [5–7].

The growing complexity and functionality of IoT devices have further complicated the processes by which vulnerabilities are triggered. To facilitate device configuration and

usage, IoT devices commonly employ a web server-based communication method to handle front-end requests and orchestrate various back-end components. This approach is particularly prevalent in Internet-connected IoT devices like routers, gateways, and cameras. Taint analysis is a widely used and effective approach for vulnerability detection in IoT firmware. This technique identifies external controllable inputs as taint sources and traces their propagation to detect simple taint vulnerabilities [8–14]. However, modern IoT devices increasingly exhibit *high-order taint vulnerabilities* [15, 16], which are particularly prevalent in complex systems such as kernels [17] and web applications [18, 19]. The triggering of high-order taint vulnerabilities involves the interaction of *multiple requests* to propagate and manipulate user-controllable input (tainted data) by *different components* of IoT firmware (details in § 2.1).

Existing taint analysis methods are inadequate for detecting high-order vulnerabilities in IoT firmware. Current techniques [9–14] primarily focus on analyzing single components or individual requests, which allow high-order taint vulnerabilities to remain concealed. For instance, in an evaluation using the SaTC dataset [20], we identified 52 0-day vulnerabilities, 25 of which are high-order vulnerabilities. Moreover, kernel-specific high-order taint analysis methods [17, 21] are also ineffective for IoT firmware due to hidden request/input parsing interfaces and the presence of inter-process communication (**IPC**) mechanisms.

The detection of high-order taint vulnerabilities in IoT firmware presents significant difficulties due to the monolithic nature of firmware, which integrates all functionalities into a single, complex system. First, such systems consist of multiple components for request handling, e.g., executable binaries with different functionalities. Second, different types of requests—identified by action keywords in request entities—are processed by distinct logic, but the action entry handler (**AEH**) corresponding to the logic is obscured. While the interface for receiving requests appears uniform, the specific AEH is concealed within custom functions and is implemented through different mechanisms such as function pointers. Third, request handling spans multiple binaries, each running as a separate process. When the AEH parses a request, its logic may transfer the request data to other processes via IPC for further processing. Fourth,

* Corresponding author.

the control dependency of shared data between requests is unclear. For instance, when a request stores data in persistent storage, the logic to retrieve this data may not execute immediately; instead, it is triggered later by other specific request handlers (details in § 2.2).

To address these challenges, we propose **Bridge**, a method designed to meet the essential requirements for high-order taint vulnerability detection in IoT firmware. **Bridge** operates in four key stages. ① *Identifying entries*: **Bridge** begins by using pattern matching within binaries to identify the AEHs for various action requests. Concurrently, it extracts keywords to link front-end and back-end features, enabling the identification of input parsing functions (IPFs) to determine taint sources. ② *Constructing cross-binary propagation dependency graphs for tainted data*: Based on the identified AEHs and taint sources, the propagation of tainted data is analyzed. During this process, IPC analysis is performed to construct the binary dependency graph (BDG), which illustrates the propagation dependencies of tainted data across the entire system. ③ *Tracing the cross-binary control dependency for tainted data IPC read and write relationships*: the explicit and implicit control relationships between binaries are analyzed based on the handlers of each action. This analysis results in the construction of action dependency graph (ADG), which represents the control dependencies between different processes. ④ *Identifying high-order taint vulnerabilities*: **Bridge** first confirms the potential taint vulnerabilities in the BDG, and then traces the control relationships through the ADG backtracking to effectively identify high-order taint vulnerabilities.

We conducted a thorough evaluation of **Bridge** based on 44 different images of real-world firmware from 8 vendors. **Bridge** discovered a total of 1,168 true positive taint vulnerabilities with a precision of 58.72%, which outperforms the SOTA Mango’s [9] 327 TPs with 11.35% precision and SaTC’s [20] 16 TPs with 4.51% precision. For high-order taint vulnerabilities, **Bridge**’s results contain 566 TPs, compared to 71 in Mango and 0 in SaTC. In addition, the high-order vulnerabilities found by Mango lack control relationships and rely on manual analysis. To date, we have assisted vendors in confirming 90 vulnerabilities (including 45 high-order vulnerabilities), which can lead to denial of service and remote code execution. In addition, we conducted comparative analysis and ablation analysis on entry identification, BDG construction, and ADG construction. The results show that **Bridge**’s design is effective and outperforms the SOTA.

In summary, we make the following contributions:

- We propose a novel graph—Action Dependency Graph (ADG)—to capture the control dependencies between action handlers in different processes. The ADG solves the difficulty of constructing different request relationships for high-order taint vulnerability detection.
- We propose **Bridge**, the first approach to detect high-order taint vulnerabilities in IoT firmware. **Bridge** implements high-order taint vulnerability analysis leveraging BDGs, generated by matching IPC static and dynamic keywords, and ADGs, constructed through the

analysis of implicit and explicit control relations.

- We conducted comprehensive evaluations on 44 real-world firmware samples, which demonstrate the effectiveness of **Bridge** in detecting high-order taint vulnerabilities. **Bridge** detects 566 high-order vulnerabilities and provides data and control dependencies, which outperforms the SOTA work Mango and SaTC.
- We publicly release the dataset, the source code of **Bridge**, and the environment configuration to facilitate collaboration and encourage further advances in the research community [22].

2. Background and Motivation

2.1. Background

2.1.1. Workflow of IoT Devices. IoT devices typically feature interfaces for external communication, enabling them to receive user configuration requests. The workflow of an IoT device can be broadly divided into two stages, as illustrated in Fig. 1, which uses a real-world example from the D-Link DIR882 Wi-Fi Router to explain the process.

Stage 1: Construct Requests. Users interact with the device through a web front-end to configure settings or query the device’s status. The web front-end generates the corresponding HTTP request and sends it to IoT devices. Each request serves a specific function, represented by distinct SOAPACTION fields. For instance, `SetNetwork` and `SetVLAN` correspond to network and VLAN configurations, respectively. These functional fields are referred to as **action keywords**, which indicate the specific action to be performed by the back-end [20]. Different action keywords also include data fields necessary for executing the associated action. For example, in Request 1 of Fig. 1, the `DIR882` with the `Devicename` tag is the request data.

Stage 2: Handling Requests. The back-end handles requests using various components, such as executable binaries, libraries, and bash scripts. The code snippets in different processes that handle specific action keyword requests are called **action handlers** [23]. The first component to process a request is called the **border binary** (e.g., `prog.cgi` in Fig. 1), which analyzes the action keyword using a registration function (e.g., `websFormDefine`) to determine the AEH, the first action handler responsible for processing the request. For instance, `AEH1` or `AEH2` in `prog.cgi` is the AEH for a specific action keyword identified by function pointers. Subsequent processing involves invoking action handlers across various processes corresponding to the action keyword to complete the request. For example, Request 2 in Fig. 1 demonstrates the intricate data and control flow relationships involved in request handling:

- *Intra- and Inter-process data flow.* Intra-process data flow refers to traditional data propagation within a single process, such as between functions or within a function. Inter-process data flow occurs when data is transferred between processes. For instance, in Fig. 1, `prog.cgi` and `rc` are separate processes.

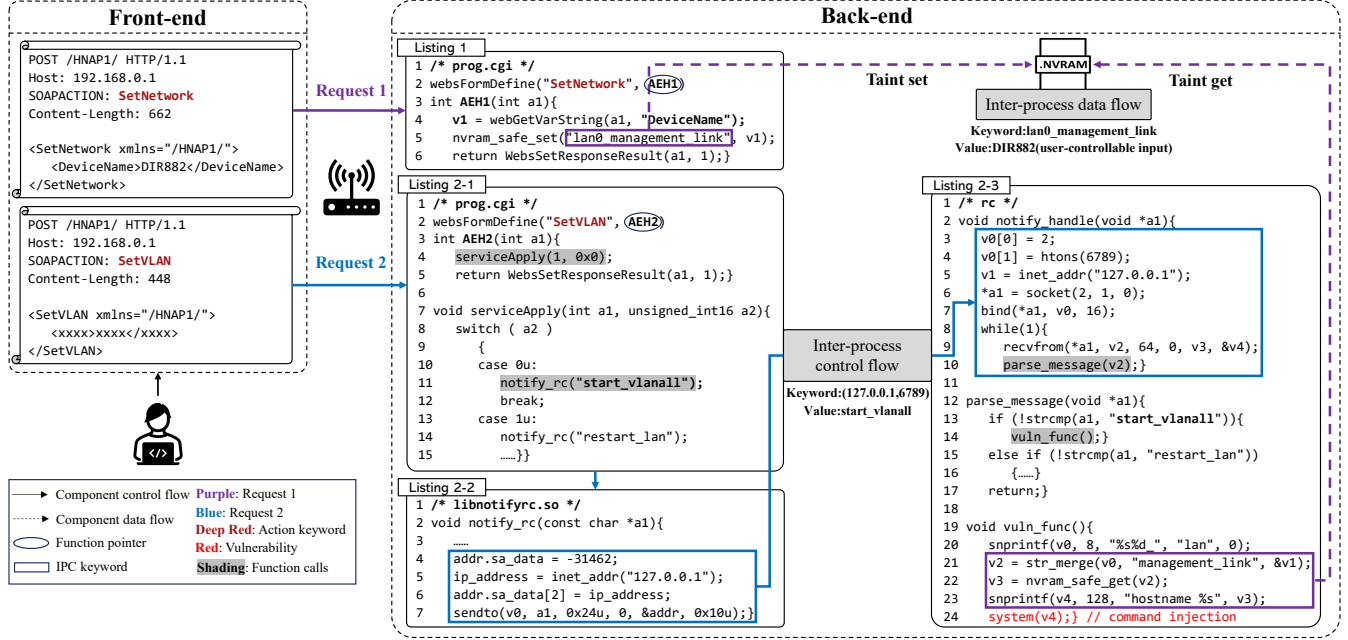


Figure 1: An example of a high-order taint vulnerability from a real IoT device. Request 1 introduces user-controllable input and Request 2 continues to propagate the tainted data, leading to a command injection vulnerability.

prog.cgi transmits data to rc via the IPC function pair nvram_safe_set/nvram_safe_get. This cross-process data transmission is termed an *IPC data message*.

- **Intra- and Inter-process control flow.** Intra-process control flow represents traditional function call relationships within a process, including direct calls, indirect calls, and library function calls. Inter-process control flow refers to control transfer between processes. For example, in Listing 2-1 of Fig. 1, prog.cgi calls the library function notify_rc, which sends data via the IPC function sendto. The corresponding recvfrom function operates within the while(1) loop of the daemon process rc. As a result, control flow transitions from line 7 in Listing 2-2 to line 10 in Listing 2-3. This cross-process control transfer is termed an *IPC control message*.

2.1.2. High-order Taint Vulnerability Definition. Our goal is to identify high-order taint vulnerabilities in IoT firmware that require the cooperation of multiple requests and are caused by the propagation of user-controllable inputs.

Consider a firmware with multiple components C_i ($i \in [1, n], n > 1$) responsible for handling requests R_i ($i \in [1, n], n > 1$). The user-controllable input contained in a request R is a taint source T . The high-order taint vulnerability H_T arises from the complex interplay of tainted data, system components, and requests, which can be represented as follows: $H_T = h(T, \{C_1, C_2, \dots, C_n\}, \{R_1, R_2, \dots, R_n\})$, where $h: \mathbb{T} \times \mathbb{C}^n \times \mathbb{R}^n \rightarrow \mathbb{H}_T$. In this formula: $\mathbb{T}$ represents the set of all user-controllable input; $\mathbb{C}^n$ represents the set of all

possible components, which process user requests, such as executable binary prog.cgi, executable binary rc, and library libnotifyrc.so; $\mathbb{R}^n$ represents the set of all possible requests, which include different action keywords and data fields; $\mathbb{H}_T$ represents the space of all possible high-order taint vulnerabilities.

The triggering of a second-order taint vulnerability generally involves the following steps:

- 1) **Introduction and Initial Storage of Tainted Data:** The first request R_1 introduces user-controllable input, which is marked as tainted data T . This data is handled by a system component C_1 and stored in a persistent storage PS_1 , such as a database, file, or NVRAM:

$$T \xrightarrow{R_1} C_1 \xrightarrow{\text{Store}(T)} PS_1$$

- 2) **Propagation of Tainted Data:** A subsequent request R_2 retrieves the T from the PS_1 . The retrieval can occur in the same component C_1 or another component C_i ($i \in [1, n]$), depending on the action keyword. After retrieval, T may be: ① Stored in another persistent storage PS_2 , waiting to be retrieved. ② Handled directly by the sink during the current request, triggering a second-order taint vulnerability.

$$PS_1 \xrightarrow{R_2} C_i (i \in [1, n]) \xrightarrow{\text{Read}(PS_1)} T \begin{cases} \xrightarrow{\text{Handle}(T)} \text{Sink} \\ \xrightarrow{\text{Store}(T)} PS_2 \end{cases}$$

If the tainted data T continues to propagate—retrieved or stored by additional requests R_i ($i \in [3, n]$) and handled by C_i ($i \in [1, n]$)—it can eventually cause a vulnerability at a sink. This process may lead to higher-order taint vulnerabilities, such as third-order vulnerabilities.

2.2. Motivation

To illustrate the challenges and motivations for detecting high-order taint vulnerabilities in IoT firmware, we present a real-world example from D-Link DIR882 (Fig. 1), which contains a 0-day vulnerability discovered by Bridge. This example demonstrates the process of triggering second-order taint vulnerabilities.

- **Request 1: Introducing Tainted Data.** The action keyword `SetNetwork` determines the AEH AEH1 to process the request based on the registration function `websFormDefine` in the border binary `prog.cgi`. Inside AEH1, the user-controllable field `DeviceName` is parsed using the input parsing function (IPF) `webGetVarString`, producing the value `DIR882`, which is stored in variable `v1`. Since `webGetVarString` parses user input, it is considered a tainted source and `v1` is marked as tainted data [24]. The IPC writing function `nvruntime_safe_set` then writes `v1` to non-volatile random-access memory (NVRAM) using the keyword `lan0_management_link`.
- **Request 2: Using Tainted Data Across Processes.** The action keyword `SetVLAN` triggers the AEH AEH2, which subsequently calls the function `serviceApply`. This function calls the library function `notify_rc`, which uses the IPC writing function `sendto` to send the message `start_vlanall` with the keywords `127.0.0.1` and `6789`. The `rc` process listens for this IPC message, stores `start_vlanall` in `v2`, and takes over the control flow. The function `parse_message` parses the IPC control message and enters the action handler `vuln_func` according to the control value `start_vlanall`. Within `vuln_func`, the IPC reading function `nvruntime_safe_get` retrieves the value `DIR882` from NVRAM, which is then passed to the system. This results in a system command injection vulnerability due to the user-controllable nature of `DIR882`.

Limitations of Existing Methods. Traditional methods fail to detect such high-order taint vulnerabilities in IoT firmware due to several aspects:

- **Hidden Communication Interfaces and Inter-Process Communications.** Methods like SUTURE [17], which focus on the Linux kernel, detect high-order taint vulnerabilities caused by system call data sharing (e.g., `read`, `write`, `open`). However, in IoT firmware, front-end and back-end interactions are common, with many back-end interfaces hidden in binaries and not directly exposed. For example, request handling and data parsing are implemented with custom functions. These hidden interfaces lead to missed external inputs, resulting in false negatives. Additionally, IoT firmware has prevalent inter-process data and control communications, and kernel analysis does not take these communications into account, leading to false negatives.
- **IPC Data and Control Dependencies.** Static analysis tools of IoT firmware, such as Karonte [25] and SaTC [20], rely on keyword matching to build IPC data dependencies. For example, they detect static keywords like

`lan0_management_link` in `nvruntime_safe_set`. However, they fail to analyze dynamically generated keywords, such as those created by the function `str_merge` in `rc` (Listing 2-3). Furthermore, these methods neglect IPC control dependencies, resulting in the inability to give the actual triggering path of vulnerability alerts. Ignoring dynamic keywords and control dependencies results in false negatives and prevents detection of true positives.

- **Exploding Input Space in Dynamic Analysis.** Dynamic analysis relies on test case execution, making it difficult to cover all code paths [26, 27]. High-order vulnerabilities require tracking the execution states of multiple requests, significantly increasing the input space. The inability to explore exhaustive scenarios leads to missed vulnerabilities, resulting in numerous false negatives.

Challenges in Detecting High-Order Taint Vulnerabilities.

The primary challenges of detecting high-order taint vulnerabilities in IoT firmware lie in identifying analysis entries, tracing the propagation of tainted data throughout the system, and elucidating the control dependencies involving IPC reading and writing function of tainted data.

- **Identifying AEHs and IPFs.** Determining AEHs and IPFs based solely on keywords shared between the front-end and back-end is unreliable [20]. On the one hand, only keywords matched in a specific control flow pattern can be used as the basis for inferring AEH (e.g., a registration function for determining AEH1), otherwise it is easy to cause false positives. On the other hand, the front-end may lack keywords to match AEH, or the front end may not exist at all, which can lead to false negatives for AEH [28]. The above problem also occurs when identifying IPF [24].
- **Handling Static and Dynamic IPC Data Dependencies.** Propagating tainted data across processes requires analyzing IPC function pairs, which involves both static and dynamically generated keyword matching. For example, the keyword `lan0_management_link` in `rc` (Listing 2-3) is generated using `str_merge`, requiring interprocedural analysis to resolve. Then `nvruntime_safe_set` in `prog.cgi` (Listing 1) can pair successfully. Existing methods [20, 25, 29] ignore dynamic keyword generation to limit search space, further contributing to false negatives.
- **Establishing Explicit and Implicit IPC Control Dependencies.** IoT firmware involves multiple binaries cooperating to handle requests, where control flow transitions between processes. Explicit relationships, such as `httpd` calling `system("sysctl -p 14")`, mean that `httpd` directly calls `sysctl`. Implicit control dependencies have no clear receiver and the control relationship is hidden in the IPC data transmission. For example, the message sent by `sendto` in Listing 2-2 and received by `recvfrom` in Listing 2-3. In addition to identifying the sender and receiver, we need to further specify the action handler based on the control value (e.g., `-p 14` and `start_vlanall`) to distinguish the handling logic of different control values and avoid false positives caused

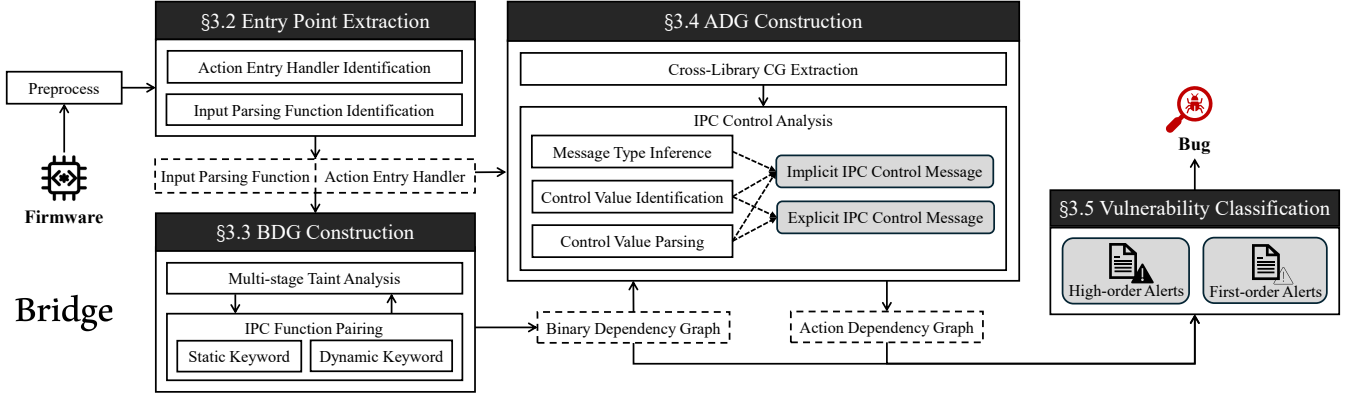


Figure 2: Workflow of Bridge.

by the same receiver.

Proposed Solution: Bridge. To address the challenges of detecting high-order taint vulnerabilities in IoT firmware, we propose **Bridge**, a novel framework that combines advanced analysis techniques to overcome the limitations of existing methods.

- **Identifying AEHs and IPFs through Pattern and Shared Keyword Matching.** Bridge first employs pattern matching to locate candidate AEHs within back-end binaries. Then, Bridge extracts the keywords in the candidate AEHs and matches them with the keywords extracted from the front-end. The back-end program where the keyword is matched successfully is regarded as the border binary. All candidate AEHs in the border binary are regarded as valid AEHs. At the same time, Bridge analyzes the call site features of each function in the border binary to extract arguments and analyzes the arguments processing logic to locate IPFs. Starting with back-end pattern matching and then using keyword matching filtering can improve the accuracy of identification. Even if the action keywords are hidden or do not exist on the front-end, Bridge can still analyze the back-end to avoid false negatives.
- **On-Demand Inter-Procedural Constant Propagation for Dynamic Keywords.** Bridge constructs the binary dependency graph (BDG) that describes the propagation path of tainted data throughout the firmware, including tainted data dependencies within and across files [25]. Starting from the IPF as the taint source, Bridge traces tainted data propagation and builds IPC data dependencies using both static and dynamic keywords. To handle dynamically generated keywords, Bridge proposes an on-demand inter-procedural constant propagation algorithm. For IPC functions, Bridge identifies call sites (e.g., line 22 in Listing 2-3) and performs precise backward data flow tracing on keyword-related arguments (e.g., `v2`). For custom functions that process these arguments, Bridge conducts inter-procedural analysis. For library functions, pre-implemented summaries are utilized. This process yields precise dynamic keywords (e.g., `lan0_management_link` for `nvram_safe_get` at line 22 in Listing 2-3). If multiple keywords reuse the

same IPC function call site, Bridge incorporates them into a unified set.

- **Action Dependency Graph (ADG) for Control Dependency Analysis.** Bridge proposes the ADG to model the control dependency between action handlers. ADG is a directed graph defined as $G = (N, E)$, where N is a set of nodes and E is a set of edges. Each node $n \in N$ is represented as a tuple $n = (b, m, k, c)$, where b is the binary name (e.g., `prog.cgi` and `rc` in Fig. 1) containing the action handler, m is the communication method (e.g., `sendto` or `recvfrom`), k is the pairing keyword (e.g., `(127.0.0.1, 6789)`), and c is the control value (e.g., `start_vlanall`). Each edge $e \in E$ is defined as $e = (n_i, n_j)$, representing control flow from n_i to n_j . The ADG construction process is to first collect the information of nodes N . Then, Bridge identifies explicit and implicit relationships based on IPC control analysis to establish edges E between nodes. At the same time, Bridge checks the control values in explicit and implicit relationships to track the relevant code logic of the specific action handler.

3. Design of Bridge

3.1. Overview

The overall workflow of Bridge is illustrated in Fig. 2. Bridge takes IoT firmware as input and begins with preprocessing, which includes extracting the file system, analyzing the file type, disassembling binaries, and collecting information such as the control flow graph (CFG), call graph (CG), and function pointer table in the `.data` segment. After preprocessing, Bridge proceeds to analyze high-order taint vulnerabilities through four key modules:

Entry Point Extraction. This module identifies Action Entry Handlers (AEHs) and Input Parsing Functions (IPFs), which are fundamental for constructing the BDG and ADG. The AEH is the entry point for external requests parsing, and the IPF is the taint source function which serves as the entry point for user-controllable data. Bridge uses a combination of pattern matching and keyword matching

to identify AEHs and IPFs. For AEHs, Bridge analyzes the CFG and CG of back-end binaries to perform pattern matching: function pointer tables, nested conditional branch selections, and registration functions. Bridge then performs matching with the front-end based on the identified action keywords of candidate AEHs to identify the border binary and determine the valid AEHs. Similarly, Bridge analyzes the parameter handling logic of each function within border binaries and extracts the call site arguments to perform front-end matching to determine the valid IPFs.

BDG Construction. This module employs a multi-stage taint analysis strategy to construct the BDG, establishing data dependencies across binaries. Bridge uses persistent storage as intermediaries during different stages of taint propagation. Specifically, the first stage uses IPFs as initial taint sources and persistent storage as sinks. In the following stages, instances of persistent storage serve as tainted sources, and new persistent storage or dangerous functions (e.g., `strcpy` and `system`) act as sinks. To identify persistent storage, Bridge pairs IPC functions with keywords (e.g., `nvramp_safe_set` and `nvramp_safe_get`). Bridge overcomes the difficulty of dynamic IPC keyword recovery via an on-demand inter-procedural constant propagation algorithm and combines with static keywords to execute IPC function pairing. Finally, the keyword and IPC function pairs enable persistent storage to support multi-stage propagation across different binaries, thereby constructing the BDG based on the read and write relationships.

ADG Construction. This module analyzes both implicit and explicit control dependencies in IPC to construct the ADG, which establishes control dependencies between action handlers. To comprehensively capture potential inter-process control relationships, Bridge constructs the cross-library CG for each binary. This graph encompasses not only the CG of the binary itself but also calls to library functions, recursively analyzed until standard libraries (e.g., `libc` and `ld`) are reached. For implicit IPC control, Bridge identifies paths involving IPC function calls within the cross-library CG and matches keywords associated with each IPC function. Bridge then analyzes the variable passed between matched IPC function pairs. If the variable influences conditional branches, the IPC messages are classified as implicit control. For explicit IPC control, the callee is determined directly based on the program name in the call instruction. Bridge further analyzes the specific arguments (control value) in control messages to locate specific processing code snippets (action handlers) and establishes control relationships between action handlers based on a producer-consumer model, forming the ADG.

Vulnerability Classification. This module tracks taint information within the BDG and ADG to identify high-order taint vulnerabilities. Bridge first analyzes the BDG to detect whether user-controllable data reaches a vulnerable sink. If a vulnerability is detected, the corresponding action handlers are located in the ADG and backtracked to identify the control dependencies that trigger the vulnerability. If triggering the vulnerability involves collaboration between multiple

```
1 //Linksys device
2 if (!strcmp(a3, "setEthPortLed")) {
3     setEthPortLed(a1, a2, v6);}
4 else if (!strcmp(a3, "langSwitch")) {
5     v49 = (_BYTE *)sub_34970(v6, "lang");
6     v50 = nvram_bufget(0, "Language");
7     if (*v49 && strcmp(v49, v50)) {
8         nvram_bufset(0, "Language", v49);}}
```

Listing 1: Nested condition.

```
1 //D-Link device
2 int ModuleInitInternet() {
3     websFormDefine("GetWanSettings", sub_45D714);
4     websFormDefine("SetWanSettings", sub_465E68);}
5 //Netgear device
6 .data:00120AB0 DCD aUpgradeCgi ; "upgrade.cgi"
7 .data:00120AB4 DCD sub_304C8
8 .data:00120AB8 DCD aFwlogCgi ; "fwLog.cgi"
9 .data:00120ABC DCD sub_2E278
```

Listing 2: Function pointer.

AEHs, it is classified as a high-order taint vulnerability; otherwise, if it only involves a single AEH and one request, it is considered a first-order taint vulnerability.

3.2. Entry Point Extraction

The entry point extraction module analyzes function features in back-end binaries, and extracts keywords to associate the front-end and back-end, ultimately determining AEHs and IPFs.

3.2.1. AEH Identification. AEHs are typically hidden in back-end binaries. Based on our observations from a large number of IoT firmware samples, numerous AEHs correspond to distinct control actions and are invoked through dispatch logic determined by control field values. This design closely resembles common practices in protocol parsers [30–32], where control fields are resolved through control tables [33] that implement control flow dispatch or event-driven handling. Such mechanisms enable the system to dynamically route requests to the appropriate handler based on a parsed field, improving efficiency and modularity. Through extensive analysis across diverse firmware samples, we find that AEHs are primarily implemented using three recurring patterns. This observation allows for the effective identification of AEHs by matching specific code patterns:

- **Nested Conditional Pattern:** Listing 1 demonstrates multi-nested `if-else` statements, which implement action keyword analysis through string comparison functions such as `strcmp` and `memcmp`, determining the specific AEH to execute.
- **Registration Function Pattern:** Lines 2-4 of Listing 2 illustrate action keyword and AEH pairing through strings and function pointers, bound together via registration functions. This pattern is commonly observed during initialization phases, characterized by frequent calls to registration functions.

Algorithm 1: Entry point extraction.

```
Input: Bins: Back-end binaries
Input: FFs: Front-end files
Output: AEHs, IPFs
1 AEHs  $\leftarrow \emptyset$  // AEH identification
2 Front-end keywords FKs  $\leftarrow$  FrontendAnalysis(FFs)
3 Border binaries BBs  $\leftarrow \emptyset$ 
4 foreach Bin  $\in$  Bins do
5   Info = dict()
6   Candidate action keyword CAKs  $\leftarrow \emptyset$ 
7   Candidate action entry handler CAEHs  $\leftarrow \emptyset$ 
8   CAKs; CAEHs  $\leftarrow$  StructurePatternAnalysis(Bin)
9   Info [Bin] = (CAKs, CAEHs)
10 foreach (Bin; CAKs; CAEHs)  $\in$  Info do
11   if KeywordMatching(CAKs, FKs) then
12     BBs.add(Bin)
13     AEHs  $\leftarrow$  AEHs  $\cup$  CAEHs
14 IPFs  $\leftarrow \emptyset$  // IPF identification
15 foreach Bin  $\in$  BBs do
16   Argstr = dict()
17   Candidate input parsing function CIPFs  $\leftarrow \emptyset$ 
18   foreach f  $\in$  Bin do
19     callers.strings  $\leftarrow$  CallSiteAnalysis(f)
20     if len(callers) > 1 and len(strings) > 0 then
21       if FunctionBehaviorMatching(f) then
22         Argstr [f] = strings
23         CIPFs.add(f)
24   f  $\leftarrow$  KeywordMatchingTopAnalysis(Argstr, FKs)
25   IPFs.add(f)
26 return AEHs, IPFs
```

- **Function Pointer Table Pattern:** Lines 6-9 of Listing 2 depict a scenario similar to the registration function pattern. Here, string pointers and function pointers are stored in the `.data` segment at fixed address offsets, forming a function pointer table. The specified AEH is invoked by resolving the action keyword index in the table.

Algorithm 1 shows the design of AEH identification. We perform control flow structure pattern matching on each binary to identify the three AEH patterns. Specifically, we analyze the CFG of each function to locate basic blocks with multiple predecessors. Then we backtrack along the predecessor paths to identify nested conditional branches involving string comparison functions (e.g., `strcmp`). The comparison strings are extracted as candidate action keywords CAKs (line 6), and the paths corresponding to these conditional branches as candidate action entry handlers CAEHs (line 7). For the registration function pattern, we identify high-frequency calls to non-standard library functions. If a function takes both string constants and function pointers as arguments, it is classified as a registration function. The corresponding strings and function pointers are then extracted as CAKs and CAEHs, respectively. For the function pointer table pattern, we scan the `.data` segment at fixed offsets to identify pairs of pointers where one is a string. Extract these pointers as CAKs and CAEHs. Based on the identified CAKs and CAEHs from each binary, AEHs are determined by cross-referencing front-end keywords FKs (line 2). If CAKs from a binary match any FKs, the binary is classified as a border binary BB (line 12), and all its CAEHs are considered AEHs.

3.2.2. IPF Identification. Bridge identifies IPFs by analyzing the behavior of functions within border binaries that are called multiple times and process string arguments. This step determines taint source functions, as described in Algorithm 1. Specifically, we first analyze the call sites of custom functions in border binaries to identify callers (line 19) and extract strings (line 19) from the arguments of these callers. We then perform reach definition analysis on qualifying functions (line 20) to analyze function behavior. Specifically, Bridge determines whether the string-containing parameters are further processed by string operation functions (e.g., `strcmp`, `strtok`, and `hsearch_r`). Bridge also checks whether the return value depends on its parameters, or whether both the return value and the parameters depend on shared global variables. If the function satisfies either of the two dependency conditions, it is classified as a candidate input parsing function CIPF (line 23). Finally, we match all *Argstr* (line 25) from CIPF call sites against front-end keywords FKs (line 25). The function with the highest number of matches is selected as the IPF.

3.3. BDG Construction

The BDG construction module analyzes static and dynamic keywords to pair IPC functions and then propagate tainted data across the entire system to establish data dependencies.

3.3.1. Multi-stage Taint Analysis. To construct the BDG, we employ a multi-stage taint propagation strategy that starts with user-controllable IPFs and propagates tainted data across the back-end. In the first stage, we use a traditional single-binary taint analysis engine [11], where the IPF serves as the taint source and IPC writing functions (e.g., `nvramp_set` or `apmib_set`) are defined as the sinks. After analyzing all IPFs, we conduct matching analysis on the tainted IPC writing functions, setting their corresponding IPC reading functions as the next-stage taint sources. In subsequent stages, we define sinks to include IPC writing functions and dangerous functions that may use tainted data to cause vulnerabilities. This process is iteratively applied across binaries in IoT firmware. By repeating taint propagation and associating binaries, Bridge constructs a BDG that represents tainted data dependencies across the back-end.

3.3.2. IPC Function Pairing. The cornerstone of BDG construction is accurately pairing IPC functions to propagate tainted data. To achieve this, we propose on-demand inter-procedural constant propagation (OD-ICP), which recovers dynamic keywords and combines them with static keywords to pair IPC functions.

For each tainted IPC writing function, Bridge extracts the IPC keyword at its call site. For constant strings or numeric constants in function arguments, the keywords are directly extracted as static keywords. For non-constant variables, we perform OD-ICP to recover dynamic keywords, leveraging both intra- and inter-procedural analyses:

TABLE 1: Constant propagation update rules.

VEX IR instructions	
Assignment	$r_j \rightarrow t_i, t_i \rightarrow r_j \text{ and } t_i \leftarrow t_j$
Binary operation	$t_i = t_n \odot t_m$ where $\odot \in \{+, -, *, /\}$
Ternary expression	$t_i \leftarrow \begin{cases} t_n, & \text{if } t_j \text{ is true} \\ t_m, & \text{otherwise} \end{cases}$ t_j is a conditional expression.
Memory Load/Store	$t_i \leftarrow \text{Mem}[t_j] \text{ and } \text{Mem}[t_j] \leftarrow t_i$
Hooked constant handling library functions	
Copy and Concatenation	strcpy, strncpy, strcat, strncat, sprintf, snprintf, memcp, strdup
Length	strlen
Conversion	atoi, itoa
Search	strchr, strstr, strtok, strtok_r, strsep

- **Intra-Procedural Analysis:** At a specific call site, we perform a pre-order traversal of basic blocks of the function containing the IPC function call. This traversal identifies constant assignments and updates. Additionally, **Bridge** hooks string manipulation and numerical computation library functions to track value updates caused by these calls. For constants generated within loops, if the loop count is known, the value is directly computed. Otherwise, a conservative assumption is made based on the initial value. The update rules applied during this analysis are summarized in Table 1.
- **Inter-Procedural Analysis:** If intra-procedural analysis cannot resolve the constant value, we analyze custom functions whose return values or arguments involve the target variable. **Bridge** performs intra-procedural analysis on these custom functions while synchronizing updates to global variables, the stack pointer, and the return value at the call site. The resolved constants are then propagated back to update the values at the original call site.

Once the tainted IPC writing function and its associated keyword are determined, **Bridge** searches for the corresponding IPC reading function in other binaries and extracts its keyword. If the keywords match, a tainted data dependency is established between the IPC writing function and the IPC reading function at their respective call sites, completing the IPC function pairing process.

3.4. ADG Construction

The ADG construction module identifies explicit and implicit control relationships between action handlers across processes.

3.4.1. Cross-Library Call Graph (CG) Extraction. To capture all potential control dependencies caused by IPC, we extract the cross-library CG for each binary and its dependent libraries. This process is analogous to taint propagation during BDG construction, as IPC functions in standard libraries are often wrapped and used by other libraries. However, cross-library CG analysis begins at an AEH and traces the call relationships until reaching a standard library function.

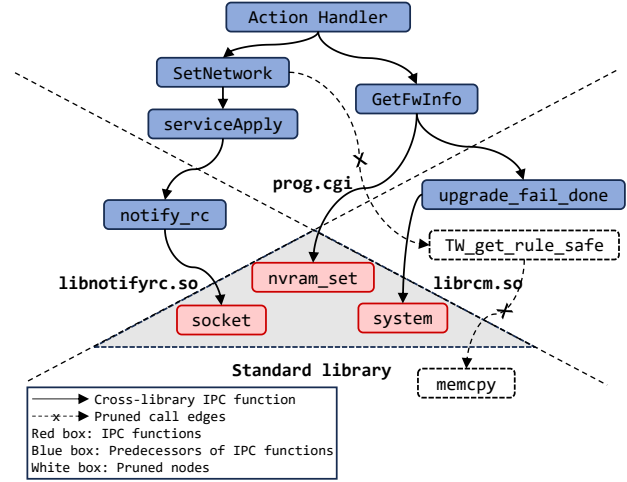


Figure 3: IPC function call graph.

The analysis initiates with an AEH in a border binary. After constructing the complete cross-library CG for a binary, we prune the paths leading to IPC function calls. For example, as illustrated in Fig. 3, paths involving `nvrnm_set`, `socket`, and `system` are retained, along with their associated nodes.

Once the IPC functions in the border binary are identified, we conduct IPC control analysis to locate calls to action handlers in other binaries (§ 3.4.2). Subsequently, the controlled action handler becomes the starting point for extracting the cross-library CG in its corresponding binary, followed by iterative pruning and IPC control analysis. Repeating this process enables the construction of the ADG for the entire IoT firmware.

3.4.2. IPC Control Analysis. IPC control analysis determines specific control dependencies among action handlers in different processes. Depending on the information conveyed in IPC messages, control relationships are categorized as explicit control or implicit control. For example, the command `system("sysctl -p 14")` explicitly directs the program to call `sysctl`. Implicit control relationships, on the other hand, are inferred from IPC data dependencies.

Message Type Inference. We infer control messages in IPC data dependencies based on their distribution patterns [34, 35]. Control messages are characterized by: (1) IPC writing functions depending on multiple control flows with identical keywords; (2) IPC reading functions calling different handlers based on specific passed values, which influence conditional branches. For instance, in Fig. 1, different values of `v2` (Lines 14 and 16) correspond to distinct branches in the code. Such behavior arises because programs send messages via a common interface, with receivers redistributing these messages to appropriate handlers based on the specific value passed.

Using the cross-library CG, we analyze the call sites of IPC writing functions across different control flows and extract both dynamic and static keywords. If identical key-

words are found in multiple control flows, we search for corresponding IPC reading functions in other binaries or libraries. A successful keyword match triggers data dependency analysis at the IPC reading function’s call site to examine the passed variable. If this variable is used in string comparisons that affect branch conditions, the IPC writing and reading functions sharing the keyword are identified as having a control dependency.

Control Value Identification. Control value identification focuses on determining specific values passed in explicit and implicit IPC controls:

- **Explicit control:** For process calls, we locate the function’s call site and extract arguments as control values, e.g., `sysctl -p 14` in `system("sysctl -p 14")`. Similarly, shell scripts are widely used in IoT firmware to propagate control values across processes [36]. These scripts typically contain conditional logic and variable assignments, which complicate direct parsing. We adopt abstract syntax tree (AST) analysis [37] to handle such structures in a structured manner. AST preserves the script’s control and data flow, allowing us to locate execution instructions (e.g., binary invocations) and trace the values of related variables. This enables accurate extraction of control values even when they are assigned or modified across multiple lines. For signal transmissions (e.g., `kill` and `raise`), we analyze the call sites of signal-sending functions and extract their arguments as control values. Furthermore, tracing the data flow of the parameter representing the target process identifier can identify its origin, such as a process identifier file, and link it to the process name.
- **Implicit control:** Starting from the root node of the IPC writing function’s call path, we employ under-constrained directed symbolic execution [38] to derive specific control values in varying call contexts. This design is suitable for recovering control values that are constructed or resolved at runtime, particularly in the presence of branching logic, parameter selection, or indirect computation. By complementing static analysis, it improves robustness in identifying control values across diverse and execution-dependent scenarios. For example, in Listing 2-2 of Fig. 1, values like `start_vlanall` and `restart_lan` can be identified. To enhance efficiency, we first conduct reverse data dependency analysis from the IPC writing function call site to determine reachable basic blocks leading to the root node. Symbolic execution is then applied selectively to relevant basic blocks, avoiding unnecessary exploration.

Control Value Parsing. The purpose of control value parsing is to map IPC writing functions and their associated code snippets (or IPC reading functions) to specific handlers. Handlers are categorized as fixed or flexible. Fixed handlers: For signal transmissions, we use the IPC value to directly identify the registered signal function (via `signal` or `sigaction`) as the handler. Flexible handlers: For implicit IPC control values and process call values, additional analysis is required to locate the handler. The command-line

arguments (e.g., `argv` in `main`) or IPC control message receiving functions (e.g., `recvfrom`) are treated as taint sources. Taint propagation is conducted to extract the tainted basic block graph within the binary. Using under-constrained symbolic execution, we analyze the control value starting from the taint source and pinpoint the branch entry influenced by the control value within the tainted basic block graph. The basic blocks affected after the branch entry are identified as the action handlers corresponding to the control value.

3.5. Vulnerability Classification

Bridge identifies high-order taint vulnerabilities by analyzing both BDGs and ADGs. BDGs record the data path of potential taint vulnerability from the source to the sink, and ADGs provide control dependencies between action handlers. For each potential taint vulnerability recorded in BDGs, we identify the IPC reading function F_{read} that the tainted data depends on within the affected binary. The corresponding IPC writing function F_{write} for F_{read} is located in another binary, based on the IPC relationship documented in BDGs. We then identify the action handlers in which F_{read} and F_{write} reside. Using ADGs, we trace back the ancestor nodes of these action handlers to their respective AEHs. This allows us to determine the AEHs that are responsible for the read and write dependencies of the tainted data.

If the AEHs associated with F_{read} and F_{write} are different, the taint vulnerability is classified as high-order, as it involves multiple distinct AEHs. When the AEHs for F_{read} and F_{write} are the same, we conduct further analysis on the CG and CFG to determine the relationship between the write and read nodes: If the read node strictly dominates the write node, the vulnerability requires two identical action keywords to trigger. In this case, the vulnerability is classified as high-order. Otherwise, the vulnerability is considered first-order.

4. Evaluation

We conduct a comprehensive evaluation of Bridge on real-world IoT devices and compare it with SOTA tools to answer the following four research questions:

- **RQ1:** How effective is Bridge in detecting high-order taint vulnerabilities?
- **RQ2:** How effective is entry point extraction?
- **RQ3:** How effective is BDG construction?
- **RQ4:** How effective is ADG construction?

4.1. Experiment Setup

Implementation. We implemented the Bridge prototype in about 11,000 lines of Python code. Bridge is able to support the analysis of executable binaries, dynamic libraries, and shell scripts in firmware for ARM, MIPS, and x86 architectures. Specifically, we first extract the firmware

TABLE 2: Taint vulnerability detection results. Alerts: the number of vulnerabilities reported by the tool. Vuls: the true positives in alerts, where the number in brackets represents high-order taint vulnerabilities. T: the time taken for cross-binary taint analysis. C: the time for constructing inter-process control dependencies.

Dataset	Vendor	Product	Bridge						Mango				SaTC			
			Alerts	Vuls(High-order)	Analyzed Bins	Time(h)			Alerts	Vuls(High-order)	Analyzed Bins	Time(h)	Alerts	Vuls(High-order)	Analyzed Bins	Time(h)
						T	C	Total								
SaTC dataset	Netgear	R7000	279	133(91)	97	1.78	6.47	8.25	360	35(17)	127	20.62	22	1	72	237.4
	Netgear	R7000P	295	138(93)	100	1.79	4.80	6.59	238	28(12)	142	24.95	33	2	75	253.56
	Netgear	R6400v2	129	69(38)	90	1.62	5.32	6.94	315	30(15)	116	17.39	51	1	66	180.66
	Netgear	XR300	127	74(46)	83	1.51	5.24	6.75	356	19(8)	113	18.9	15	1	64	187.59
	Netgear	R8500	115	65(29)	84	1.57	3.73	5.3	238	19(2)	107	14.41	17	2	58	176.64
	D-Link	DIR878	65	33(4)	36	0.61	2.09	2.7	20	5	70	11.14	1	1	15	45.11
	D-Link	DIR882	57	39(6)	31	0.53	2.00	2.53	45	13(1)	64	10.11	1	1	14	43.17
	D-Link	DIR823G	2	2(1)	34	0.66	0.84	1.5	0	0	0	5.28	0	0	0	0.57
	Tenda	G1G3	8	7(4)	18	0.41	0.45	0.86	16	3	57	3.3	27	0	19	45.47
	TOTOLink	T10	2	2	44	0.54	2.52	3.06	0	0	32	4.39	0	0	9	27.25
	TOTOLink	A950RG	4	4	52	0.54	1.19	1.73	0	0	35	8.3	0	0	11	35.21
New dataset	Motorola	MR2600	62	41(6)	26	0.49	1.89	2.38	49	11(1)	56	12.05	2	2	9	44.57
	Trendnet	TEW820	16	12(4)	34	0.52	1.43	1.95	0	0	52	7.9	0	0	5	33.57
	Linksys	RE6500	87	77(23)	56	0.85	1.56	2.41	4	2	63	2.77	3	0	18	43.98
	Linksys	RE9000	65	57(5)	51	0.78	1.72	2.5	3	2	59	2.34	5	3	16	49.15
	Linksys	E1700	48	44(8)	41	0.74	1.55	2.29	2	2	71	6.03	0	0	7	45.27
	Linksys	MR7350	0	0	0	0	0	0	12	0	115	16.99	0	0	11	1.76
	Netgear	R6400	84	35(17)	80	1.53	4.97	6.5	137	20(6)	97	15.08	49	0	62	178.1
	Netgear	EX6200	97	63(50)	38	0.83	1.19	2.02	102	13(1)	43	4.93	34	1	26	167.44
	Netgear	WNDR4500	24	22(9)	44	1.05	1.02	2.07	225	32(3)	80	16.16	0	0	30	68.24
	Netgear	MR62	41	9(4)	122	1.78	5.50	7.28	200	3(1)	173	25.4	3	0	96	181.56
	Netgear	CBR40	47	17(17)	76	1.25	4.79	6.04	36	5(1)	111	13.02	8	1	46	85.21
	Netgear	BBR750	88	31(22)	81	1.35	8.04	9.39	89	10(3)	70	10.02	2	0	68	73.84
	D-Link	DIR1935	62	32(4)	37	0.57	2.44	3.01	44	10	65	16.17	0	0	12	69.93
	D-Link	DIR816	64	58(43)	18	0.39	0.86	1.25	5	5	33	3.84	0	0	1	0.72
	D-Link	DIR619L	7	7(5)	34	0.45	1.04	1.49	0	0	0	0.22	0	0	0	0.85
	D-Link	DIR605L	7	7(5)	32	0.43	0.84	1.27	0	0	0	0.20	0	0	0	0.81
	D-Link	DCS932L	11	4(4)	19	0.51	0.58	1.09	3	0	24	1.65	0	0	0	0.7
	TOTOLink	T6T8	2	2(2)	40	0.68	1.27	1.95	0	0	38	4.74	0	0	1	1.14
	TOTOLink	A720R	1	1	36	0.20	0.71	0.91	0	0	51	8.4	0	0	0	0.78
	TOTOLink	A3002R	6	6(5)	43	0.61	2.77	3.38	0	0	37	6.54	0	0	0	1.25
	TOTOLink	X5000R	15	15(4)	25	0.37	1.48	1.85	60	12	41	5.65	1	0	10	9.69
	TOTOLink	LR350	12	12(4)	21	0.41	1.16	1.57	59	8	30	4.2	0	0	10	8.34
	TOTOLink	NR1800X	23	23(4)	26	0.42	1.25	1.67	78	17	40	4.6	1	0	11	9.77
	TOTOLink	A3700R	15	15(4)	29	0.36	1.40	1.76	73	14	45	4.08	1	0	12	8.27
	TOTOLink	EX1200T	1	1	37	0.46	1.23	1.69	0	0	20	0.23	0	0	6	20.7
	Tenda	AX3	6	5(5)	26	0.46	0.54	1	8	1	69	7.92	39	0	37	100.39
	Tenda	AX12	0	0	17	0.16	0.87	1.03	6	0	39	2.58	0	0	0	0.92
	ASUS	RT-AX56	11	2	85	0.97	7.86	8.83	45	1	149	11.5	16	0	102	164.92
	ASUS	RT-AC68U	1	1	62	0.85	5.03	5.88	31	1	112	23.36	25	0	96	168.85
	ASUS	RT-AC1200GU	2	2	23	0.36	2.86	3.22	10	3	53	6.23	0	0	15	4.11
	ASUS	RT-N53	1	1	21	0.35	0.39	0.74	13	3	36	7.37	0	0	7	4.68
Total		44	1989	1168(566)	1949	31.74	102.89	134.63	2882	327(71)	2735	390.96	355	16(0)	1117	2782.14
Average		\	45.2	26.55(12.86)	44.3	0.72	2.34	3.06	65.5	7.43(1.61)	62.16	8.86	8.07	0.36(0)	25.39	63.23

filesystem based on Binwalk [39] and load the analysis object using IDA Pro [40]. In this process, we extract CFG and CG, and lift the assembly code to VEX IR to facilitate the analysis of different architectures. We then perform taint propagation based on the single-binary taint analysis engine EmTaint [11] to construct the BDG. The ADG is constructed using symbolic execution with angr [41].

Dataset. We collected two datasets totaling 44 real-world firmware images to evaluate **Bridge**. **SaTC dataset.** We used the public vulnerability detection dataset of SaTC [20], which includes 12 firmware images from 4 vendors. Although SaTC performed well on parts of this dataset, the dataset is outdated, and vendors no longer accept related vulnerability disclosures. Therefore, we used the latest versions of these firmware images for additional 0-day vulnerability analysis, vulnerability verification and disclosure to vendors. **New dataset.** We further expanded the dataset by device types, series, and vendors to build a new dataset that contains 32 latest version firmware images from 8 vendors.

Evaluation Environment. All experiments were performed on a Linux server equipped with dual Intel Xeon Silver 4210 CPUs and 128GB of RAM.

4.2. High-order Taint Vulnerability Detection-RQ1

This section evaluates the efficacy of **Bridge** in discovering high-order taint vulnerabilities on the real-world firmware dataset. Furthermore, we compare **Bridge** with

the SOTA works **Mango** [9] and **SaTC** [20] for detecting cross-binary taint vulnerabilities in IoT firmware. We comprehensively analyze the results and verify true positives (TPs) using the same criteria adopted by previous studies [20, 25]. A TP is considered when user-controllable data affects a specific IPC keyword and is used by a sink without sanitization, leading to an overflow or command injection. When the reading and writing of TP tainted data involve different requests (§ 3.5), it is considered a high-order taint vulnerability.

4.2.1. Effectiveness. We manually verified alerts by debugging real devices. Table 2 shows that **Bridge** issued a total of 1989 alerts, of which 1168 are TPs. The 1168 TPs include 566 high-order taint vulnerabilities and 602 first-order taint vulnerabilities.

Bridge's analysis of MR7350 failed because its firmware is based on Lua implementation (discussed in § 5). Moreover, we analyzed false positives (FPs) of **Bridge** and found that FPs are all caused by insufficient sanitization during the taint propagation. The large number of customized functions in the firmware lead to over-contamination caused by inadequate sanitizer settings, which is a common problem in firmware analysis. Fortunately, the precision of **Bridge** shown in Fig. 4a is 58.72%, which illustrates that the analysis of control relationships (ADG) has alleviated the occurrence of FPs.

Vulnerability Validation. We manually verified vulner-

TABLE 3: 0-day vulnerabilities discovered by Bridge.

Vendor	Product	Type	High-order Vulnerability	First-order Vulnerability
Motorola	MR2600	CI	CVE-2022-34885	3 unassigned
Linksys	RE6500	BoF	2 unassigned	CVE-2025-8824, CVE-2025-8826
			CVE-2025-8816, CVE-2025-8817	CVE-2025-8831, CVE-2025-8832
		CI	CVE-2025-8819, CVE-2025-8820	CVE-2025-8825
D-Link	DCS932L	BoF	CVE-2025-8822	CVE-2025-8825
			CVE-2025-8818, CVE-2025-8821	CVE-2025-8827 to CVE-2025-8830
		CI	CVE-2025-8823	\
D-Link	DIR816	BoF	CVE-2025-5572	\
		CI	CVE-2025-5573	\
TOTOLink	X5000R	BoF	CVE-2025-5620	CVE-2025-5621, CVE-2025-5622
			1 unassigned	\
		CI	3 unassigned	CVE-2021-45736
TOTOLink	T6/T8	CI	\	CVE-2022-26213
			CVE-2022-25133	CVE-2022-27003, CVE-2022-27004
Netgear	R7000/R7000P	BoF	CVE-2024-51015, CVE-2024-51020	CVE-2024-51003
			CVE-2024-51017 to CVE-2024-51019	\
		BoF	CVE-2024-50996, CVE-2024-51011	CVE-2024-50997, CVE-2024-51002
Netgear	R6400v2	BoF	CVE-2024-51007, CVE-2024-51014	\
			CVE-2024-51021, CVE-2024-51022	\
		CI	\	CVE-2024-51008, CVE-2024-51010
Netgear	R8500	BoF	PSV-2022-0174, PSV-2022-0175	CVE-2024-51006, CVE-2024-51012
			PSV-2022-0180, CVE-2024-50994	\
		CI	CVE-2024-50995, CVE-2024-50998	\
D-Link	DIR878	CI	CVE-2024-51000, CVE-2024-51001	PSV-2022-0062, PSV-2022-0082
			CVE-2024-51009	PSV-2022-0083, PSV-2022-0193
		CI	CVE-2021-44882	CVE-2022-27947, CVE-2024-50993
D-Link	DIR882	CI	CVE-2024-48631, CVE-2024-48634	CVE-2024-50999, CVE-2024-51005
			\	CVE-2021-44880
		CI	\	CVE-2024-48635 to CVE-2024-48638
D-Link	DIR823G	CI	CVE-2024-51023	CVE-2024-48629, CVE-2024-48630
			\	CVE-2024-48632, CVE-2024-48633
		CI	1 unassigned	CVE-2024-51024
TOTAL	\	\	45	45

CVE: the vulnerability IDs assigned by CNAs.

PSV: the vulnerability IDs assigned by Netgear.

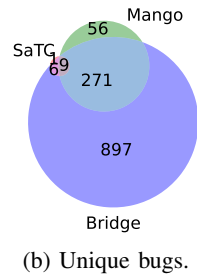
Unassigned: the vendor has confirmed the vulnerability but needs to fix it before releasing details.

abilities (TPs/FPs) by debugging real devices, including firmware version upgrades/downgrades. This process involved four experienced analysts working hundreds of hours, with an average of 15 minutes spent per alert.

Vulnerability Disclosure. We responsibly disclosed TPs to the vendors. So far, 90 0-day vulnerabilities have been confirmed or assigned CVE/PSV numbers (Table 3). Among them, 45 are high-order taint vulnerabilities that can lead to DOS and RCE. Of the 90 0-day vulnerabilities confirmed, 49 were completed by us in collaboration with the corresponding vendors. Due to the long duration of vulnerability disclosure, there are 41 vulnerabilities that D-Link and Netgear have not addressed because the devices are at the end-of-life status. We reported these 41 0-day vulnerabilities through VulDB [42] and MITRE [43] after confirming the vulnerabilities were not covered by NVD [44] entries and that the proof-of-concept can trigger them.

Metrics	Bridge	Mango	SaTC
Alerts	1989	2882	355
TP	1168	327	16
FP	825	2555	339
Precision	58.72%	11.35%	4.51%

(a) Vulnerability precision.



(b) Unique bugs.

Figure 4: Comparison with Mango and SaTC.

Comparison to SOTA. Table 2 shows that Mango issued 2882 alerts of which 327 are TPs and SaTC issued 355 alerts of which 16 are TPs. Bridge’s precision of 58.72% is better than Mango’s 11.35% and SaTC’s 4.51%,

as shown in Fig. 4a. Moreover, Bridge discovered 897 unique vulnerabilities as shown in Fig. 4b. For high-order taint vulnerabilities, we reconstructed inter-process control dependencies and provided them to the baseline tools. With these reconstructed dependencies, Mango reported 71 cases, while SaTC reported none. However, they lack the ability to explicitly output such relations, and the reported vulnerabilities are buried in a large number of FPs. This highlights Bridge’s unique advantage in discovering high-order vulnerabilities.

The main reasons why Bridge outperforms Mango and SaTC are as follows. Pattern matching combined with shared keyword analysis to locate IPF identifies hidden taint sources (see § 4.3 for details). The recovered dynamic IPC keywords increase the effectiveness of IPC taint propagation (see § 4.4 for details). The construction of ADG enables high-order vulnerability control flow recovery (see § 4.5 for details). Specifically, Mango performs reverse taint tracing from sinks, which often yield many FPs from non-user-controlled data. When reverse taint analysis reaches a custom function, if its string parameter matches a front-end string or comes from an NVRAM, Mango identifies it as a vulnerability. However, this is a FP when the NVRAM value is not actually tainted by user input, or when the matched front-end string is merely an unrelated static value rather than a user-controllable input (IPF). Furthermore, Mango’s design leads to a high number of FNs. On the one hand, Mango does not consider dynamic IPC keywords that disrupt cross-binary taint propagation. On the other hand, Mango relies solely on front-end/back-end string sharing without pattern matching, thus missing hidden taint sources. For SaTC, it relies on a symbolic execution engine to conduct taint analysis, which makes SaTC prone to path explosion, while its scalability is further challenged when analyzing multi-binary firmware, and it suffers from FN issues similar to those of Mango.

We further analyzed the FNs of Bridge in Fig. 4b. For 1 of SaTC, Bridge failed to track a tainted heap offset pointer. For 56 of Mango, Bridge did not handle dynamic keywords generated by cumulative values in loops. This is because the loop control variable comes from an external definition that cannot be traced. Although Mango can analyze this situation by performing reverse taint analysis at any sink, this strategy increases FPs and lowers precision.

4.2.2. Efficiency. In terms of time overhead, the average analysis time of Bridge is 3.06 hours, which includes the end-to-end time from the firmware preprocessing to the alert generation. As shown in Table 2, the average analysis time of Bridge is 2.9 times faster than Mango and 20.7 times faster than SaTC. The time advantage comes from Bridge’s accurate extraction of entry points and IPC keywords, and its on-demand expansion of analysis to other binaries based on inter-process control and data flows, which together avoid redundant program space exploration and significantly reduce analysis time. In contrast, Mango performs reverse taint tracing from every sink point in each binary, even though many binaries are not involved in the propagation or trig-

gering of user-controlled taint, resulting in a large amount of redundant analysis. SaTC, on the other hand, relies on a heavyweight symbolic execution engine, which makes scalable taint analysis difficult, as a significant amount of time is wasted on path exploration and constraint solving.

4.3. Entry Point Extraction-RQ2

This section evaluates the efficacy of entry point extraction. We analyzed the identification precision of AEH and IPF, and also used border binary as an indicator, which was analyzed by SaTC. Mango was not included in the comparison because Mango uses reverse taint analysis which does not require identification of entry points. There is no ground truth for firmware entry point identification, so we verify the correctness of the analysis results. We use dynamic debugging of firmware to verify the correctness of static analysis results, which is an effective and reliable method [45–47]. We manually construct requests to trigger function execution and collect firmware execution information to determine the analysis results.

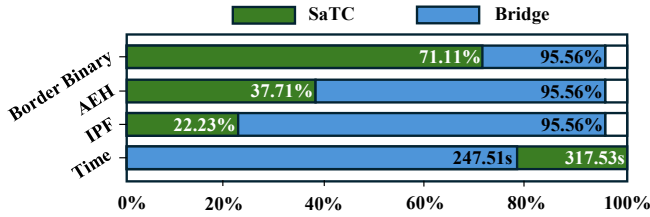


Figure 5: Comparison of entry point extraction.

Fig. 5 shows that Bridge’s identification precision for border binary, AEH, and IPF is 95.56%, 95.56%, and 95.56%, respectively, which are all better than SaTC’s 71.11%, 37.71%, and 22.23%. In addition, Bridge’s average time overhead is 77.95% of SaTC’s. We analyzed the reasons for the improvement in results, which mainly come from the strategy of combining pattern matching and keyword filtering, which can identify hidden AEHs and IPFs while filtering FPs. Bridge identified an average of 48.53 hidden AEHs and 117.16 hidden IPFs per firmware (there is no front-end or the keywords do not exist in the front-end), whereas SaTC did not find any hidden interfaces (detailed results in Appendix B).

4.4. BDG Construction-RQ3

This section evaluates the effectiveness of Bridge’s BDG construction and compares it with SaTC. Moreover, we conduct ablation studies to illustrate the contribution of IPFs and dynamic IPC keywords to BDG construction.

4.4.1. Completeness. The effectiveness of taint vulnerability detection directly depends on the completeness of BDG construction. We compared the completeness of Bridge’s BDG with that of SaTC. As shown in Table 4, the average number of nodes and edges in Bridge’s BDG is 45 and

44 times greater than those in SaTC’s BDG, respectively. For each firmware sample, the BDG of Bridge is more complete than that constructed by SaTC (detailed results in Appendix C).

TABLE 4: Comparison of BDG construction completeness.

	Ave. Nodes	Ave. Edges
SaTC	18.16	40.86
Bridge	808.52(↑ 45×)	1780.32 (↑ 44×)

To verify whether the IPC paths recovered by Bridge are valid, we randomly selected two firmware samples: D-Link DIR-878 and Netgear R7000. The results show that about 94% of the IPC paths involved in cross-binary taint propagation identified by Bridge are correct. The remaining inaccuracies mainly stem from OD-ICP’s (§ 3.3.2) lack of modeling of certain non-standard string processing functions in custom libraries, which can be addressed with some additional minor engineering effort by adding summaries for custom library functions that share the same semantics as standard library routines.

4.4.2. Ablation Study. This completeness improvement of the BDG is attributed to the IPF identification and dynamic IPC keywords. We randomly selected 18 firmware samples for ablation study to determine the impact of different strategies on BDG construction and vulnerability discovery. Specifically, we set up the following three configurations for comparison:

- **Naive:** Using IPFs identified by SaTC and the static keyword matching strategy of SaTC.
- **IPF:** Using IPFs identified by Bridge and the static keyword matching strategy of SaTC.
- **OD-ICP:** Using IPFs identified by SaTC and the keyword matching strategy (static and dynamic) of Bridge.

The left vertical axis of Fig. 6 shows that enabling IPF and OD-ICP both contribute to the construction of BDGs. IPF provides more entry points, and OD-ICP increases the success rate of IPC function pairing. Furthermore, the

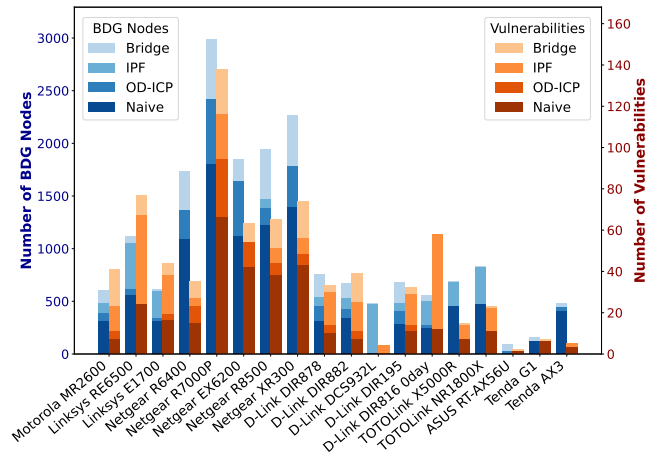


Figure 6: The ablation results of BDG construction.

right vertical axis of Fig. 6 shows that the completeness of BDGs directly benefits the detection of vulnerabilities. For example, in the D-Link DCS932L, hidden entry points prevent Naive from discovering any vulnerabilities.

4.5. ADG Construction-RQ4

This section illustrates the effectiveness of Bridge’s ADG construction by analyzing the precision of ADG results and comparing them with the only related work, Chkup [45]. Chkup contains an inter-process control relationship analysis module to analyze the firmware update path. We then conduct ablation studies to illustrate the contribution of three types of inter-process control dependencies to high-order taint vulnerabilities.

4.5.1. Effectiveness. We randomly selected 18 firmware samples for ADG precision evaluation. We manually verified whether each inferred inter-process control dependency truly exists at runtime. For every edge in the ADG, we dynamically debugged the relevant processes by triggering the producer-side command and observing whether the expected consumer process was activated. By setting breakpoints at action handlers and monitoring IPC activities (e.g., socket communication or shared storage access), we determined whether the inferred dependency was correct (detailed results in Appendix D). The summary results are shown in Table 5. Bridge identifies 509, 2346, and 677 TPs of implicit IPC, process call, and signal control dependencies, respectively, and their precision rates are all over 84%. However, Chkup can only identify 4 process call dependencies. This is because Chkup only captures process calls within the update logic code and does not perform implicit relationship analysis and specific parameter analysis.

TABLE 5: Comparison of ADG construction precision.

	Implicit IPC			Process call			Signal		
	TP	FP	Precision	TP	FP	Precision	TP	FP	Precision
Chkup	0	0	0	4	0	100%	0	0	0
Bridge	509	62	89.14%	2346	419	84.85%	677	121	84.84%

We further analyzed FPs in ADGs. First, the calculation of some control values of process calls and signals caused tracing failures due to the complex calculation and deep call propagation involved. In addition, during the resolution of control values, the symbolic execution was aborted due to timeout. The timeout is an effective mitigation measure to avoid the analysis of real firmware falling into path explosion.

4.5.2. Ablation Study. We analyze the control dependencies involved in triggering high-order tainted vulnerabilities to illustrate the necessity of the three control dependencies. Table 6 shows the number of vulnerabilities involving implicit IPC, signals, and process calls as 20, 37, and 167, respectively. We observed that both implicit control dependencies and explicit control dependencies are widely present in devices from various vendors. However, vendors

implement data security verification to different degrees, so the vulnerabilities are not necessarily distributed in every vendor’s device. In addition, there are 342 vulnerabilities that do not involve inter-process control relations, that is, both the reading and writing of tainted data occur in the border binary. However, the triggering of these vulnerabilities also requires the construction of multiple requests.

TABLE 6: The contribution of inter-process control flow to the discovery of high-order taint vulnerabilities.

Vendor	Vuls	AEHs	Implicit IPC	Signal	Process call
Netgear(11)	416	252	0	37	127
D-Link(8)	72	43	14	0	15
Linksys(4)	36	22	0	0	14
Motorola(1)	6	0	6	0	0
TOTOLink(11)	23	15	0	0	8
Tenda(4)	9	8	0	0	1
Trendnet(1)	4	2	0	0	2
ASUS(4)	0	0	0	0	0
Total(44)	566	342	20	37	167

5. Discussion

Scalability of Bridge. Bridge analyzes the back-end implemented in C/C++ and shell scripts in Linux-based firmware. As for programming languages, Bridge did not consider back-end written in other languages such as PHP or Lua. However, compared with C/C++ and shell scripts, other languages are less commonly used or even unused [48]. In addition, Bridge’s ideas can also be implemented through engineering work to analyze other languages.

For systems, Bridge performs analysis on Linux-based firmware, which is a common system used in IoT devices with complex functionality [46, 49]. Other embedded systems, such as bare-metal firmware or real-time operating system firmware, are often used in scenarios with high real-time requirements and simple functionality, such as baseband chips and programmable logic controllers [50, 51]. Moreover, Bridge relies on dynamically linked binaries with symbol information for accurate static analysis. Dynamic linking is common in Linux-based firmware due to storage constraints [52]. For stripped binaries or static links without symbol tables, code similarity analysis can effectively recover the standard library function symbol table [53–55], so code similarity analysis can be used to expand the application scope of Bridge.

Application Scope. The applicability of Bridge is shaped by both its AEH identification strategy and its system-level assumptions. In terms of AEH identification, Bridge currently relies on three specific programming patterns, which are derived from our study of mainstream vendor systems. While static analysis inevitably incurs some omissions, once the entry points are identified—either automatically or manually—the subsequent construction of the BDG/ADG proceeds fully automatically. Furthermore, Bridge operates in a web-based front-end/back-end workflow model. This model is common in Linux-based IoT devices [10, 12, 20, 25, 56]. For other types of IoT architectures, such as multi-tier

app–cloud–device systems [57] Bridge currently cannot handle these scenarios. A promising direction is to enhance Bridge with cross-layer analysis that correlates companion-app logic and cloud-side APIs with firmware entry points. Once such cross-layer mappings are available, Bridge can leverage its BDG/ADG construction to enable end-to-end detection of high-order taint vulnerabilities spanning app, cloud, and firmware layers.

Indirect Call. Indirect call resolution remains an open problem. Bridge leverages the algorithm in EmTaint [11] to handle indirect calls (e.g., function pointers). However, highly complex indirect control transfers remain challenging (e.g., callback pointers returned from external modules or opaque virtual dispatch via vptr mutation / layout abstraction) [58–61], and the imprecision in resolving such calls can affect the completeness of BDG/ADG construction. Bridge will benefit from more advanced solutions to indirect call resolution in the future, which are orthogonal to our work [62–64].

Manual Verification. The results of static taint analysis need to be manually verified, and Bridge is no exception. We verified the results by purchasing the corresponding real device and adopting dynamic debugging, which resulted in heavy manual work. Therefore, we believe that automated verification of static analysis results is a direction worth exploring, especially when static analysis involves large-scale analysis scenarios.

6. Related Work

High-order taint vulnerability analysis. High-order taint vulnerability detection has made initial progress in complex systems such as kernels and web applications. SUTURE [17] considers global variables as taint relays and adopts a summary-based approach to enumerate high-order taint flows in operating system kernel source code. Dahse [18] and Olivo [19] focused on detecting second-order vulnerabilities in web applications by using database entries as taint relays. However, the data and control dependencies caused by the complex and diverse IPC in IoT firmware make existing studies ineffective. In addition, closed-source binaries in IoT firmware and request handlers hidden in the binary further exacerbate the analysis challenges. Bridge addresses the above challenges and implements the first high-order taint vulnerability detection for IoT firmware.

Static analysis of IoT firmware. Static taint analysis is a mainstream technique for detecting vulnerabilities in IoT firmware. Most studies [11–14, 24, 65] focus on analyzing vulnerabilities in single binaries in IoT firmware, that is, only analyzing border binaries that directly receive user input, which results in false negatives of vulnerability. If these studies are used to analyze other binaries in IoT firmware, a large number of false positives will be generated because the input source may be user-uncontrollable or sanitized by border binaries. Karonte [25] first introduced the concept of multi-binary interaction and used symbolic execution to analyze IoT firmware. Since then, SaTC [20],

Mango [9], and OctopusTaint [10] further optimized the cross-binary taint analysis algorithm, such as identifying intermediate taint sources and performing reverse data flow analysis to improve taint propagation efficiency. However, these approaches all ignore high-order taint vulnerabilities caused by interactions between different requests, which involve not only cross-binary taint data dependencies but also cross-binary control dependencies of taint data. When these control dependencies are caused by different requests, the increase in input space makes it difficult to track the triggering of high-order taint vulnerabilities. Therefore, Bridge fills the gap by detecting high-order taint vulnerabilities through the analysis of BDGs and ADGs.

Dynamic analysis of IoT firmware. Dynamic analysis of IoT firmware is performed through black-box fuzzing on real devices or grey-box fuzzing on firmware emulation [66]. Black-box fuzzing [67–70] suffers from low efficiency and poor code coverage, which is further exacerbated by the state between requests in high-order vulnerabilities. Grey-box fuzzing [49, 56, 71–74] is limited by the fidelity of firmware emulation environment. Existing studies rely on heavy manual work to meet the emulation conditions, but the emulation success rate is still low and the emulation environment is prone to crashing and exiting during the test. Yu Lei [27] implemented dynamic taint analysis by tracing the request flow, but the huge input space prevented the analysis results from covering inter-process behaviors, and therefore failed to identify high-order taint vulnerabilities. High-order vulnerabilities involve multiple request collaborations and data processing across binaries. Such a huge input space makes dynamic methods ineffective.

7. Conclusion

This paper presents Bridge, a novel method for detecting high-order taint vulnerabilities in IoT firmware. Bridge addresses the limitations of existing taint analysis techniques, which often fail to account for the complexities of modern IoT firmware. By constructing BDGs and ADGs, Bridge successfully tracks tainted data propagation and control dependencies across multiple components and processes, enabling the detection of vulnerabilities that would otherwise remain undetected. Extensive evaluations on real-world IoT firmware demonstrate the efficacy of Bridge. Bridge detected a total of 1,168 TPs, significantly outperforming state-of-the-art solutions like Mango, which detected only 327 TPs, and SaTC, which detected just 16. For high-order taint vulnerabilities, Bridge discovered 566, far exceeding Mango’s 71 and SaTC’s inability to identify any. These results underscore Bridge’s superior capability in addressing high-order taint vulnerabilities and mark a significant advancement in the field of IoT firmware security.

Acknowledgments

We are grateful to our shepherd and the anonymous reviewers for their valuable guidance and insightful comments. This research is supported by the project Network

Twin Enhancement Method (No. E3Z0111104) and the National Key Research and Development Program of China (No. 2022YFB3103904).

References

- [1] Statista, "Internet of things (iot) - statistics and facts," <https://www.statista.com/topics/2637/internet-of-things/#topicOverview>, 2024, accessed: 2025-05-06.
- [2] T. Verge, "Ring cameras suffered a hacking incident," <https://www.theverge.com/2020/1/6/21050426/ring-control-center-privacy-dashboard-app-police-security-two-factor-ces-2020>, 2020, accessed: 2025-05-06.
- [3] TidBITS, "Eufy cameras uploaded footage to cloud," <https://talk.tidbits.com/t/eufy-home-security-cameras-caught-uploading-footage-to-the-cloud/20619>, 2023, accessed: 2025-05-06.
- [4] T. Intelligence, "Vulnerability roundup: April 10, 2024," <https://blog.talosintelligence.com/vulnerability-roundup-april-10-2024/>, 2024, accessed: 2025-05-06.
- [5] A. H. Association, "The food and drug administration reports potential cybersecurity risk in insulin pump system," <https://www.aha.org/news/headline/2022-09-20-fda-reports-potential-cybersecurity-risk-insulin-pump-system>, 2022, accessed: 2025-05-06.
- [6] M. Dive, "vulnerability in cardiac data workflow system," <https://www.medtechdive.com/news/MDT-Medtronic-cybersecurity-vulnerability-CISA/654480/>, 2023, accessed: 2025-05-06.
- [7] Vicone, "Exploring a zero-click rce vulnerability in tesla's tpms," <https://vicone.com/blog/under-pressure-exploring-a-zero-click-rce-vulnerability-in-teslas-tpms>, 2023, accessed: 2025-05-06.
- [8] F. Yamaguchi, A. Maier, H. Gascon, and K. Rieck, "Automatic inference of search patterns for taint-style vulnerabilities," in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 797–812.
- [9] W. Gibbs, A. S. Raj, J. M. Vadayath, H. J. Tay, J. Miller, A. Ajayan, Z. L. Basque, A. Dutcher, F. Dong, X. Maso *et al.*, "Operation mango: Scalable discovery of taint-style vulnerabilities in binary firmware services," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 7123–7139.
- [10] A. Qasem, M. Debbabi, and A. Soeanu, "Octopustaint: Advanced data flow analysis for detecting taint-based vulnerabilities in iot/iiot firmware," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2355–2369.
- [11] K. Cheng, Y. Zheng, T. Liu, L. Guan, P. Liu, H. Li, H. Zhu, K. Ye, and L. Sun, "Detecting vulnerabilities in linux-based embedded firmware with sse-based on-demand alias analysis," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 360–372.
- [12] Z. Gao, C. Zhang, H. Liu, W. Sun, Z. Tang, L. Jiang, J. Chen, and Y. Xie, "Faster and better: Detecting vulnerabilities in linux-based iot firmware with optimized reaching definition analysis," in *Proceedings of the 2024 Network and Distributed System Security Symposium, San Diego, CA, USA*, vol. 26, 2024.
- [13] X. Yin, R. Cai, X. Zhu, Q. Yang, E. Song, and S. Liu, "Precise discovery of more taint-style vulnerabilities in embedded firmware," *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [14] K. Cheng, Q. Li, L. Wang, Q. Chen, Y. Zheng, L. Sun, and Z. Liang, "Dtaint: detecting the taint-style vulnerability in embedded device firmware," in *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2018, pp. 430–441.
- [15] "CVE-2019-7297: A second-order command injection vulnerability in the D-Link DIR-823G device," <https://nvd.nist.gov/vuln/detail/CVE-2019-7297>, 2019, accessed: 2025-05-06.
- [16] "CVE-2023-33239: A second-order command injection vulnerability in the Moxa TN-4900/TN-5900 firmware," <https://nvd.nist.gov/vuln/detail/CVE-2023-33239>, 2023, accessed: 2025-05-06.
- [17] H. Zhang, W. Chen, Y. Hao, G. Li, Y. Zhai, X. Zou, and Z. Qian, "Statically discovering high-order taint style vulnerabilities in os kernels," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 811–824.
- [18] J. Dahse and T. Holz, "Static detection of second-order vulnerabilities in web applications," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 989–1003.
- [19] O. Olivo, I. Dillig, and C. Lin, "Detecting and exploiting second order denial-of-service vulnerabilities in web applications," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 616–628.
- [20] L. Chen, Y. Wang, Q. Cai, Y. Zhan, H. Hu, J. Linghu, Q. Hou, C. Zhang, H. Duan, and Z. Xue, "Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 303–319.
- [21] Z. Liang, X. Zou, C. Song, and Z. Qian, "K-leak: Towards automating the generation of multi-step infoleak exploits against the linux kernel," in *31st Annual Network and Distributed System Security Symposium, NDSS*, 2024.
- [22] J. Peng, P. Liu, Y. Zeng, K. Cheng, Y. Liu, Y. Yang, and H. Zhu, "Bridge-artifact-evaluation," Zenodo, 2025, artifact for IEEE S&P 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.17232400>
- [23] B. Yuan, Z. Song, Y. Jia, Z. Lu, D. Zou, H. Jin, and L. Xing, "Mqtactic: Security analysis and verification for logic flaws in mqtt implementations," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 2385–2403.
- [24] P. Liu, Y. Zheng, C. Sun, C. Qin, D. Fang, M. Liu, and L. Sun, "FITS: Inferring Intermediate Taint Sources for Effective Vulnerability Analysis of IoT Device Firmware," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2023, pp. 138–152.
- [25] N. Redini, A. Machiry, R. Wang, C. Spensky, A. Continella, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1544–1561.
- [26] Y. Zhang, W. Huo, K. Jian, J. Shi, H. Lu, L. Liu, C. Wang, D. Sun, C. Zhang, and B. Liu, "Srfuzzer: an automatic fuzzing framework for physical soho router devices to discover multi-type vulnerabilities," in *Proceedings of the 35th annual computer security applications conference*, 2019, pp. 544–556.
- [27] L. Yu, L. Li, H. Wang, X. Wang, H. He, and X. Gong, "Towards automated detection of higher-order memory corruption vulnerabilities in embedded devices," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1707–1710.
- [28] "D-Link DCS-932L Day/Night Cloud Camera," <https://www.dlink.com/uk/en/products/dcs-932l-day-night-cloud-camera>, 2016, accessed: 2025-05-06.
- [29] P. Liu, D. Fang, C. Qin, K. Cheng, S. Lv, H. Zhu, and L. Sun, "Finding vulnerabilities in internal-binary of firmware with clues," in *ICC 2022 - IEEE International Conference on Communications*, 2022, pp. 5397–5402.
- [30] R. D. Graham and P. C. Johnson, "Finite state machine parsing for internet protocols: Faster than you think," in *2014 IEEE Security and Privacy Workshops*. IEEE, 2014, pp. 185–190.
- [31] L. Cojocar, J. Zaddach, R. Verdult, H. Bos, A. Francillon, and D. Balzarotti, "PIE: Parser identification in embedded system," in *Proceedings of the 31st Annual Computer Security Applications Conference*, 2015, pp. 251–260.
- [32] Z. Luo, F. Zuo, Y. Jiang, J. Gao, X. Jiao, and J. Sun, "Polar: Function code aware fuzz testing of ics protocol," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 1–22, 2019.
- [33] W. contributors, "Control table and its structure," https://en.wikipedia.org/wiki/Control_table, 2025, accessed: 2025-05-06.
- [34] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Communications Magazine*, vol. 51, no. 2, pp. 114–119, 2013.
- [35] B. E. Ujcich, S. Jero, R. Skowyra, S. R. Gomez, A. Bates, W. H. Sanders, and H. Okhravi, "Automated discovery of cross-plane event-based vulnerabilities in software-defined networking," in *Network and Distributed System Security Symposium*, 2020.
- [36] H. Li, Q. Huang, F. Ding, H. Hu, L. Cheng, G. Gu, and Z. Zhao,

- “Understanding and detecting remote infection on linux-based iot devices,” in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, 2022, pp. 873–887.
- [37] “Bashlex,” <https://github.com/idank/bashlex>, 2024, accessed: 2025-05-06.
- [38] D. A. Ramos and D. Engler, “Under-constrained symbolic execution: Correctness checking for real code,” in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 49–64.
- [39] “Binwalk,” <https://github.com/ReFirmLabs/binwalk>, 2024, accessed: 2025-05-06.
- [40] “IDA Pro: The Interactive Disassembler,” <https://hex-rays.com/ida-pro>, 2024, accessed: 2025-05-06.
- [41] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel *et al.*, “Sok:(state of) the art of war: Offensive techniques in binary analysis,” in *2016 IEEE symposium on security and privacy (SP)*. IEEE, 2016, pp. 138–157.
- [42] “VulDB - The Number One Vulnerability Database,” <https://vuldb.com/>, 2025, accessed: 2024-05-06.
- [43] “MITRE Corporation,” <https://www.mitre.org/>, 2025, accessed: 2025-05-06.
- [44] “National Vulnerability Database (NVD),” <https://nvd.nist.gov/>, 2025, accessed: 2025-05-06.
- [45] Y. Wu, J. Wang, Y. Wang, S. Zhai, Z. Li, Y. He, K. Sun, Q. Li, and N. Zhang, “Your firmware has arrived: A study of firmware update vulnerabilities,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 5627–5644.
- [46] A. Costin, A. Zarras, and A. Francillon, “Automated dynamic firmware analysis at scale: a case study on embedded web interfaces,” in *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*, 2016, pp. 437–448.
- [47] H. Xiao, Z. Wei, J. Dai, B. Li, Y. Zhang, and M. Yang, “Housefuzz: Service-aware grey-box fuzzing for vulnerability detection in linux-based firmware,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2025, pp. 3507–3525.
- [48] “The best embedded programming languages for engineers now,” <https://www.benigo.com/the-best-embedded-programming-languages-for-engineers-now>, 2024, accessed: 2025-05-06.
- [49] H. J. Tay, K. Zeng, J. M. Vadayath, A. S. Raj, A. Dutcher, T. Reddy, W. Gibbs, Z. L. Basque, F. Dong, Z. Smith *et al.*, “Greenhouse: Single-service rehosting of linux-based firmware binaries in user-space emulation,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 5791–5808.
- [50] Y. Liu, C. Zhang, F. Li, Y. Li, J. Zhou, J. Wang, L. Zhan, Y. Liu, and W. Huo, “Semantic-enhanced static vulnerability detection in base-band firmware,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.
- [51] E. López-Morales, U. Planta, C. Rubio-Medrano, A. Abbasi, and A. A. Cardenas, “Sok: Security of programmable logic controllers,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 7103–7122.
- [52] H. Zhang, M. Ren, Y. Lei, and J. Ming, “One size does not fit all: security hardening of mips embedded systems via static binary debloating for shared libraries,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 255–270.
- [53] X. Jin, K. Pei, J. Y. Won, and Z. Lin, “Symlm: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1631–1645.
- [54] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, “Debin: Predicting debug information in stripped binaries,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1667–1680.
- [55] Z. L. Chua, S. Shen, P. Saxena, and Z. Liang, “Neural nets can learn function type signatures from binaries,” in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 99–116.
- [56] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, “Firmae: Towards large-scale emulation of iot firmware for dynamic analysis,” in *Proceedings of the 36th Annual Computer Security Applications Conference*, 2020, pp. 733–745.
- [57] K. Liu, M. Yang, Z. Ling, Y. Zhang, C. Lei, J. Luo, and X. Fu, “Riotfuzzer: Companion app assisted remote fuzzing for detecting vulnerabilities in iot devices,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2341–2354.
- [58] X. Fan, Y. Sui, X. Liao, and J. Xue, “Boosting the precision of virtual call integrity protection with partial pointer analysis for c++,” in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2017, pp. 329–340.
- [59] K. Lu and H. Hu, “Where does it go? refining indirect-call targets with multi-layer type analysis,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1867–1881.
- [60] A. Pawlowski, V. van der Veen, D. Andriesse, E. van der Kouwe, T. Holz, C. Giuffrida, and H. Bos, “Vps: excavating high-level c++ constructs from low-level binaries to protect dynamic dispatching,” in *Proceedings of the 35th Annual Computer Security Applications Conference*, 2019, pp. 97–112.
- [61] T. Xia, H. Hu, and D. Wu, “Deeptype: Refining indirect call targets with strong multi-layer type analysis,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 5877–5894.
- [62] S. H. Kim, C. Sun, D. Zeng, and G. Tan, “Refining indirect call targets at the binary level,” in *NDSS*, 2021.
- [63] R. Sun, Y. Guo, Z. Wang, and Q. Zeng, “Attnccall: Refining indirect call targets in binaries with attention,” in *European Symposium on Research in Computer Security*. Springer, 2023, pp. 391–409.
- [64] Y. Shi, L. Tian, L. Chen, Y. Yang, and G. Shi, “Scheduled execution-based binary indirect call targets refinement,” in *European Symposium on Research in Computer Security*. Springer, 2024, pp. 3–23.
- [65] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, Z. Xu, Z. Li, P. Di, Y. Jiang, and L. Sun, “Llm-powered static binary taint analysis,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Feb. 2025. [Online]. Available: <https://doi.org/10.1145/3711816>
- [66] P. Liu, Y. Zheng, C. Sun, H. Li, Z. Li, and L. Sun, “Battling against protocol fuzzing: Protecting networked embedded devices from dynamic fuzzers,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 4, Apr. 2024. [Online]. Available: <https://doi.org/10.1145/3641847>
- [67] J. Chen, W. Diao, Q. Zhao, C. Zuo, Z. Lin, X. Wang, W. C. Lau, M. Sun, R. Yang, and K. Zhang, “Iotfuzzer: Discovering memory corruptions in iot through app-based fuzzing,” in *NDSS*, 2018.
- [68] N. Redini, A. Continella, D. Das, G. De Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, and G. Vigna, “Diane: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 484–500.
- [69] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, and Y. Xiang, “Snippuzz: Black-box fuzzing of iot firmware via message snippet inference,” in *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, 2021, pp. 337–350.
- [70] H. Liu, S. Gan, C. Zhang, Z. Gao, H. Zhang, X. Wang, and G. Gao, “Labrador: Response guided directed fuzzing for black-box iot devices,” in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2024, pp. 127–127.
- [71] J. Zaddach, L. Bruno, A. Francillon, D. Balzarotti *et al.*, “Avatar: A framework to support dynamic security analysis of embedded systems’ firmwares,” in *NDSS*, vol. 14, no. 2014, 2014, pp. 1–16.
- [72] D. D. Chen, M. Woo, D. Brumley, and M. Egele, “Towards automated dynamic analysis for linux-based embedded firmware,” in *NDSS*, vol. 1, 2016, pp. 1–1.
- [73] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, “Firm-aft: High-throughput greybox fuzzing of iot firmware via augmented process emulation,” in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1099–1114.
- [74] Y. Zheng, Y. Li, C. Zhang, H. Zhu, Y. Liu, and L. Sun, “Efficient grey-box fuzzing of applications in linux-based iot devices via enhanced user-mode emulation,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 417–428.

TABLE 8: Comparison of BDG construction completeness.

Vendor	Product	SaTC		Bridge	
		Nodes	Edges	Nodes	Edges
Netgear	R7000	10	7	3131	9056
Netgear	R7000P	10	7	2993	8549
Netgear	R6400v2	10	7	1904	2988
Netgear	XR300	6	3	2269	3587
Netgear	R8500	10	7	1948	3109
D-Link	DIR878	43	210	755	1281
D-Link	DIR882	116	312	672	1111
D-Link	DIR823G	0	0	38	28
Tenda	G1	0	0	161	125
Tenda	G3	0	0	161	125
TOTOLINK	T10	0	0	285	825
TOTOLINK	A950RG	0	0	343	2218
Motorola	MR2600	76	277	607	1063
Trendnet	TEW820	0	0	386	784
Linksys	RE6500	93	155	1115	1793
Linksys	RE9000	113	164	530	940
Linksys	E1700	102	105	614	822
Netgear	R6400	10	7	1740	2645
Netgear	EX6200	0	0	1852	5717
Netgear	WNDR4500	0	0	328	350
Netgear	MR62	4	2	1511	1922
Netgear	CBR40	6	3	2368	7212
Netgear	RBR750	6	3	2650	7573
D-Link	DIR1935	173	521	679	1213
D-Link	DIR816	8	5	556	818
D-Link	DIR619L	0	0	373	672
D-Link	DIR605L	0	0	387	706
D-Link	DCS932L	0	0	484	370
TOTOLink	T6/T8	0	0	303	678
TOTOLink	A720R	0	0	172	581
TOTOLink	A3002R	0	0	387	478
TOTOLink	X5000R	0	0	691	1102
TOTOLink	LR350	0	0	583	964
TOTOLink	NR1800X	0	0	832	1372
TOTOLink	A3700R	0	0	786	1203
TOTOLink	EX1200T	0	0	352	3758
Tenda	AX3	0	0	485	489
Tenda	AX12	0	0	9	7
ASUS	RT-AX56U	2	2	88	67
ASUS	RT-AC68U	1	1	35	26
ASUS	RT-AC1200GU	0	0	5	3
ASUS	RT-N53	0	0	7	4
Average	\	18.16	40.86	808.52(45 × ↑)	1780.32 (44 × ↑)

in-depth analysis even in IoT firmware lacking front-end files, leveraging heuristic methods.

Appendix C.

Detailed Results of BDG Construction

The detailed results of comparison of BDG construction completeness are shown in Table 8. It showcases the experimental results for all the devices in our dataset.

Appendix D.

Detailed Results of ADG Construction

The detailed results of ADG construction are shown in Table 9. In this section, we analyze the causes of high-order vulnerabilities based on the statistical results of ADG construction and elaborate on some details of the ADG construction process.

Our analysis shows that 60% of high-order vulnerabilities are triggered entirely within border binaries, without involving complex inter-process control flows. This aligns with common IoT firmware behavior, where stored user configurations are reloaded during browser interactions. These vulnerabilities are especially prevalent in Netgear devices.

TABLE 9: Comparison of ADG construction completeness.

Vendor	Product	Bridge				Chkup			
		Implicit IPC		Process call		Signal		Process call	
		TP	FP	TP	FP	TP	FP	TP	FP
Netgear	R7000	0	0	247	60	109	27	2	0
Netgear	XR300	0	0	206	45	111	19	0	0
Netgear	EX6200	0	0	89	28	42	7	0	0
Netgear	CBR40	0	0	96	38	74	13	2	0
D-Link	DIR878	69	8	45	19	35	7	0	0
D-Link	DIR823G	0	0	45	11	32	0	0	0
D-Link	DIR1935	70	8	63	16	32	8	0	0
D-Link	DCS932L	0	0	68	4	8	0	0	0
TOTOLink	X5000R	81	10	40	9	5	1	0	0
TOTOLink	LR3505	76	9	27	6	4	1	0	0
TOTOLink	NR1800X	86	11	56	12	4	1	0	0
TOTOLink	T6/T8	0	0	227	33	85	0	0	0
Linksys	RE6500	0	0	474	27	25	11	0	0
Linksys	E1700	0	0	478	35	6	11	0	0
Motorola	MR2600	63	3	21	12	29	6	0	0
ASUS	RT-AX56	64	13	60	28	5	6	0	0
Trendnet	TEW820	0	0	81	26	70	1	0	0
Tenda	G1/G3	0	0	23	10	1	2	0	0
Total		18	509	62	2346	419	677	121	4

The remaining 40% involve inter-process control flow coordination. Among them, 20 vulnerabilities are associated with implicit IPC control dependencies, primarily observed in firmware from D-Link, Motorola, TOTOLink, and ASUS. Such dependencies often mediate control transitions from border binaries to daemons. Since request handling in IoT firmware is distributed across processes, failing to resolve the initial implicit IPC link can prevent recovery of the entire control flow chain.

Explicit dependencies, such as process calls, play a major role in vulnerability propagation and are used across most vendors. However, for control flow transfer via signals, although implemented by various manufacturers, only Netgear exhibits inter-process control paths that trigger vulnerabilities.

In the process of generating the ADG, we employed several strategies to avoid unnecessary analysis. For instance, after identifying the consumer process of a particular IPC control message, we performed a pre-analysis of the corresponding file for that process. If the following conditions are met, the file is considered an endpoint file and is excluded from the ADG construction:

- The binary does not contain any calls to IPC functions.
- The binary does not access tainted persistent storage.

The first condition ensures that the binary does not perform additional control flow transformations, while the second ensures that the binary does not propagate or trigger taint vulnerabilities. Additionally, during command parsing, we exclude commands found in `busybox` or `coreutils`, as these binaries typically do not involve the propagation or triggering of taints.

Appendix E.

Design Details and Limitations

E.1. Taint Analysis

Static taint analysis is known to suffer from two inherent issues: over-tainting and under-tainting. Bridge addresses

these challenges with design choices at both the BDG and ADG construction phases.

In the BDG, we employ domain-/object-sensitive propagation to limit the spread of taint and alleviate over-tainting by avoiding pollution of unrelated program states. For the ADG construction phase, we explicitly account for implicit flows, which are a major source of under-tainting. Command-line arguments (e.g., `argv` in the `main`) or IPC control message receivers (e.g., `recvfrom`) are designated as taint sources. The analysis then focuses on tainted branch conditions and comparison operations. To capture implicit flows, **Bridge** adopts a two-phase tainting strategy. In the first phase, we automatically collect assignment statements inside basic blocks influenced by tainted branches. At present, we mainly consider cases where the assigned value is a constant, since this is the most common form of implicit flow in practice. In the VEX IR, such constant assignments are typically represented by `PUT` and `Store` instructions. In the second phase, these collected instructions are reintroduced as taint sources, allowing indirect control dependencies to be propagated further. By combining the tainted branch conditions from both phases with the call graph, we identify the functions containing such implicit flows and their corresponding call paths.

Despite these strategies, limitations remain. In the BDG, taint propagation precision can still be affected by complex aliasing or incomplete function summaries, which may lead to residual over-tainting. In the ADG, while our two-phase strategy mitigates many under-tainting cases, more complex scenarios may still be missed—for example, multi-level implicit flows where the outcome of one tainted branch influences another control decision through several intermediate computations, or non-constant data-dependent stores that cannot be safely modeled as taint sources. These residual challenges reflect the inherent trade-offs of static taint analysis, and addressing them would require more advanced pointer analysis and path-sensitive techniques beyond the current scope of **Bridge**.

E.2. Symbolic Execution

Symbolic execution is known to suffer from an inherent challenge: path explosion. **Bridge** employs three strategies to mitigate path explosion. (1) Search Space Pruning: **Bridge** performs lightweight static data-dependency analysis (backward slicing from control-related instructions) to restrict the search space to code regions involved in IPC calls. It then applies under-constrained, directed symbolic execution so that exploration focuses only on basic blocks and functions influencing control decisions, while functions irrelevant to control-value generation are not explored and instead directly return an unconstrained symbolic value, preventing exploration of any code paths that lack data/control dependence on the target control values. (2) Function Summarization: To avoid unnecessary exploration of complex library functions, we extend `angr` with function summaries for routines relevant to control-value resolution, such as the `getopt` family, specific string-processing functions,

and network address conversion functions, while ensuring that their semantic effects on program state can still be accurately captured. (3) Timeout Mechanism: We enforce a per-execution timeout (5 minutes) to prevent infinite or excessively deep explorations from blocking the analysis pipeline. Despite these optimizations, approximately 10% of executions still time out, which can affect the completeness of ADG construction. In addition, pruning remains highly dependent on the precision of static analysis algorithms, and handling more complex cases will require more advanced optimizations.

Appendix F. Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

F.1. Summary

This paper introduces Bridge, a novel system for detecting "high-order" taint vulnerabilities in Linux-based IoT firmware. These vulnerabilities are those that require multiple, coordinated request actions to be triggered. Bridge operates in three stages: it first identifies entry points and taint sources, then constructs a Binary Dependency Graph (BDG) to model interactions and data flow across different binaries, and finally builds an Action Dependency Graph (ADG) to analyze the control dependencies between action handlers. The system was evaluated on 44 real-world firmware images, where it reportedly outperformed state-of-the-art tools by identifying a large number of vulnerabilities, including 90 new vulnerabilities that were confirmed by vendors.

F.2. Scientific Contributions

- Provides a valuable step forward in an established field.
- Creates a new tool to enable future science.
- Provides a new dataset for public use.

F.3. Reasons for Acceptance

- 1) The paper provides a valuable step forward in an established field. It tackles the important and challenging problem of identifying complex, multi-step vulnerabilities in IoT firmware, which are often missed by existing tools. The primary reason for acceptance is the demonstrated effectiveness of the proposed tool, Bridge, which found a substantial number of real-world vulnerabilities (566 high-order vulnerabilities, 90 of which were vendor-confirmed) that escaped prior state-of-the-art approaches. The ablation studies also effectively demonstrate the contribution of the novel BDG and ADG components.
- 2) The authors have committed to making their tool and the dataset of firmware images publicly available, which will be a valuable resource for enabling future research in this area.

F.4. Noteworthy Concerns

- 1) Generality and Scope: The approach relies heavily on a set of three programming patterns to identify Action Entry Handlers (AEHs) and assumes a common web-based front-end/back-end workflow. While the authors provided evidence in the rebuttal that these patterns

are prevalent in routers and other common IoT devices and discussed the application scope in Section 5, this dependency limits the tool's generality.

- 2) Formal Modeling and Conceptual Clarity: One reviewer found the formal modeling ineffective and imprecise. Additionally, the core term "high-order taint vulnerability" was criticized for conflating a detection technique with a security property violation.

Appendix G. Response to the Meta-Review

We sincerely thank the reviewers for their constructive feedback and the Shepherd for the thoughtful guidance throughout the process. In light of the concerns raised in the meta-review, we would like to offer the following clarifications and explanations.

- 1) We conducted a broad study of mainstream vendors' firmware and analyzed AEH implementations from over 200 CVE-related firmware samples collected in the past two years. Our results show that the three identified patterns are sufficient to cover IoT firmware in practice. A rare case was found where a daemon invokes multiple CGI files for further processing, which is already naturally captured by our ADG. Although purely static pattern matching may still miss uncommon cases and remains an open challenge, the subsequent BDG and ADG analyses are fully automated once entry points are identified. Looking ahead, we plan to adopt LLM-based techniques to further enhance the generalizability of our static analysis. The assumption of a web-based front-end/back-end workflow is reasonable, as IoT devices such as routers, range extenders, and cameras commonly provide web-based interfaces for configuration and control. Web services have become a standard component in IoT firmware, offering convenient user interaction while also constituting the primary attack surface in real-world devices.
- 2) In firmware vulnerability analysis, the primary research focus lies in designing effective automated detection methods. Accordingly, the core objective of Bridge is not to propose a new formalism, but to address high-order taint vulnerabilities in IoT devices by leveraging existing taint analysis principles together with our BDG/ADG structures for automated detection. We use the term "high-order taint vulnerability" purely as an engineering-oriented description of the detection target, referring to vulnerabilities triggered by interactions between multiple requests (for example, one write followed by a subsequent read), rather than as a redefinition of security properties. The principle of taint analysis remains to mark taint sources, track their propagation, and detect potential hazards at sinks, such as when a tainted parameter reaches a system call. Bridge unifies taint propagation with inter-process data and control flows through BDG/ADG, thereby improving the coverage and efficiency of automated high-order taint vulnerability detection in IoT firmware.